



TOC (Barsha Pokharel) - computer engineering notes

Computer engineering (Nepal Engineering College)



Scan to open on Studocu

Finite state machine (FSM):

A FSM is defined as:

$$M = (S; I, O, F, G)$$

where

S = finite set of states

I = finite set of inputs

O = finite set of outputs

F = state relⁿ or transition fns

G = output relⁿ

- A state relⁿ is of the form -
(current state, input) $\rightarrow$ next state
- A output relⁿ is of the form -
(current state, input) $\rightarrow$ (next, state, output)

eg: ON/OFF switch as a FSM.

$S = \{ON, OFF\}$

$I = \{press\}$

$O = \{fan ON, fan OFF\}$

F consists of:

(ON, press) $\rightarrow$ OFF

(OFF, press) $\rightarrow$ ON

G consists of:

(ON, press) $\rightarrow$ (OFF, fan OFF)

(OFF, press) $\rightarrow$ (ON, fan ON)

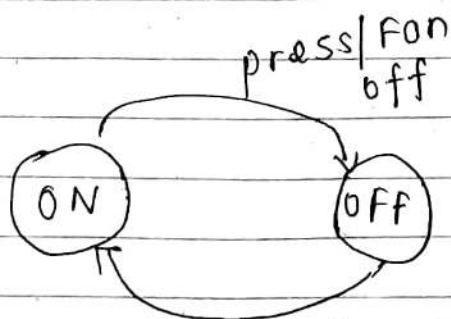


Fig: state diagram or transition diagram

	0	1
0	00	10
1	01	11

Finite state automata:

Alphabet:

- Collection of input symbols
- It is denoted by Σ

eg: binary alphabet, $\Sigma = \{0, 1\}$

Strings:

- combination or multiple occurrence of input symbols.
- It is denoted by w
- eg $\Sigma = \{0, 1\}$, $w = 11010$

Language:

- collection of all possible string over some given alphabet.
- It is denoted by L .
- eg: $\Sigma = \{0, 1\}$, $L = \{0, 1, 00, 01, 010, 1101, \dots\}$

Finite state automata (FSM without output)

A finite state automata (FSA) is a mathematical model that is used to determine whether a particular string is valid or invalid. If a string is valid then it is said to be accepted, otherwise rejected. Therefore, a FSA is also called language recognizer.

Mathematically, a FSA is defined as:

$$M = (Q, \Sigma, \delta, q_0, F)$$

where,

Q = finite set of states

Σ = finite sets of inputs

δ = transition fn, that takes two argument in the form of state input and gives an argument in the form of state. A transition fn is of the form:

$$Q \times \Sigma \rightarrow Q$$

q_0 = initial or start state

F = set of final or accepting states

eg: $Q = \{q_1, q_2, q_3\}$

$\Sigma = \{0, 1\}$

δ consists of:

$$\delta(q_1, 0) \rightarrow q_2 \quad \delta(q_2, 0) \rightarrow q_3 \quad \delta(q_3, 0) \rightarrow q_2$$

$$\delta(q_1, 1) \rightarrow q_1 \quad \delta(q_2, 1) \rightarrow q_1 \quad \delta(q_3, 1) \rightarrow q_3$$

$q_0 = q_1$

$F = q_2$

Representation of FSA:

1. Transition diagram
2. Transition table

Transition diagram:

- directed, weighted graph.
- states are represented by vertices & transition from one state to another is represented by a directed edge from one vertex to another.
- Inputs are represented by the weights provided to each edge.
- Initial state is represented by pointing an arrow towards it.
- Final state is represented by double or concentric circle.

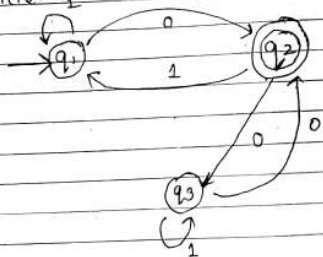


Fig: transition diagram

Transition table:-

- It is the tabular representation of FSA.
- Generally, states are arranged in rows and inputs are arranged in columns.
- Each cell of the table represents next state.
- Initial state is represented by pointing an arrow towards it.
- final state is represented by enclosing it in a circle or placing star (asterisk) sign in front of it.

Q \ Σ	0	1
$\rightarrow q_1$	q_2	q_1
* q_2	q_3	q_1
q_3	q_2	q_3

Fig: Transition table

Processing of a string by FSA.

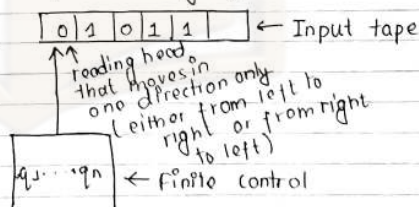


Fig: Block diagram of a FSA

A FA consists of:

1. Input tape
2. Reading head
3. Finite control

Input tape of a FA is divided into number of square cells and each cell is responsible for holding the symbols of input string. Reading head is responsible for reading the contents of each cell at a time in designed direction i.e. either from left to right or from right to left.

Finite control is used to control the processing of FA. In the initial state, intermediate states must be the states designed in finite control. Initially, the reading head is placed at the leftmost cell of the input tape. Then, it reads each cell at a time from left to right. We cross that option consuming a symbol, FA regularly changes its states. During the process of consuming inputs symbols when an empty string is encountered i.e. string terminates i.e. finite control gives any of the designed final states as next state, the string is said to be accepted otherwise rejected.

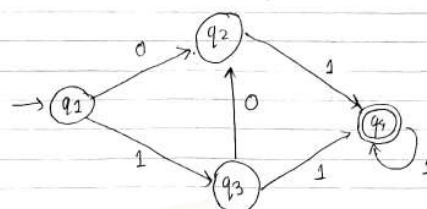


Fig: Finite automata

Check whether the string $w = 100011$ is accepted by the above FA or not.

$$\begin{aligned}
 \delta(q_1, 100011) &\rightarrow \delta(q_3, 00011) \\
 &\rightarrow \delta(q_2, 0011) \\
 &\rightarrow \delta(q_2, 011) \\
 &\rightarrow \delta(q_2, 11) \\
 &\rightarrow \delta(q_4, 1) \\
 &\rightarrow \delta(q_4, \epsilon)
 \end{aligned}$$

Since, q_4 is the final state
 $w = 100011$ is accepted.

eg: $w = 1000$

$$\begin{aligned}
 \delta(q_1, 1000) &\rightarrow \delta(q_3, 000) \\
 &\rightarrow \delta(q_2, 00) \\
 &\rightarrow \delta(q_2, 0) \\
 &\rightarrow \delta(q_2, \epsilon)
 \end{aligned}$$

Since, q_2 is not a final state $w = 1000$ is rejected.

Types of finite automata:

1. Deterministic finite automata (DFA)
2. Non-deterministic finite automata (NFA)

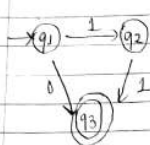


Fig: DFA

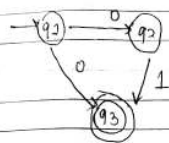
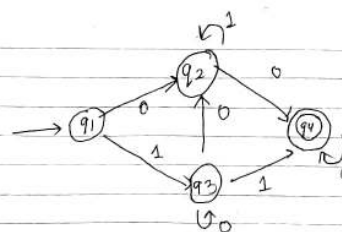


Fig: NFA



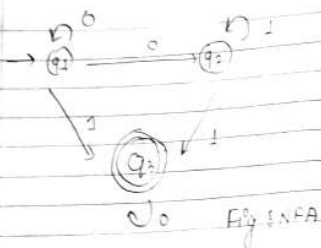
$w = 10001$

Conversion of NFA to DFA:

To convert given NFA to its equivalent DFA, we use sub-set construction method.

Steps:

- 1) Construct all the subsets by using all the states of given NFA.
- 2) Construct the transition table of the given NFA.
- 3) Transition table gives next states as old states and new states.
- 4) Identify transitions for all the new states in terms of given inputs.
- 5) If a new state generates further new state then identify transition for this new state as well.
- 6) Continue the process until transitions for all new states are identified.
- 7) Finally, draw a transition diagram.



All possible sets are: $\{q_1\}$, $\{q_2\}$, $\{q_3\}$, $\{q_1, q_2\}$, $\{q_1, q_3\}$, $\{q_2, q_3\}$, $\{q_1, q_2, q_3\}$

The transition table of given NFA is

q \ F	0	1
$\rightarrow q_1$	$\{q_1, q_2\}$	$\{q_3\}$
q_2	$\{q_2\}$	$\{q_3\}$
q_3	$\{q_3\}$	$\{q_3\}$

Let δ' be the transition fn of the DFA being formed

$$\delta'(\{q_1\}, 0) \rightarrow \delta(q_1, 0) \rightarrow \{q_1, q_2\} \text{ - new state}$$

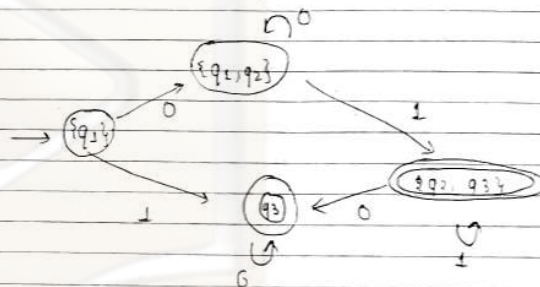
$$\delta'(\{q_1\}, 1) \rightarrow \delta(q_1, 1) \rightarrow \{q_3\} \text{ - old state}$$

$$\begin{aligned} \delta'(\{q_1, q_2\}, 0) &\rightarrow \delta(q_1, 0) \cup \delta(q_2, 0) \\ &\rightarrow \{q_1, q_2\} \cup \{q_1\} \\ &\rightarrow \{q_1, q_2\} \end{aligned}$$

$$\begin{aligned} \delta'(\{q_1, q_2\}, 1) &\rightarrow \delta(q_1, 1) \cup \delta(q_2, 1) \\ &\rightarrow \{q_3\} \cup \{q_3\} \\ &\rightarrow \{q_3\} \text{ - new state} \end{aligned}$$

$$\begin{aligned} \delta'(\{q_2, q_3\}, 0) &\rightarrow \delta(q_2, 0) \cup \delta(q_3, 0) \\ &\rightarrow \{q_2\} \cup \{q_3\} \\ &\rightarrow \{q_2, q_3\} \end{aligned}$$

$$\begin{aligned} \delta'(\{q_2, q_3\}, 1) &\rightarrow \delta(q_2, 1) \cup \delta(q_3, 1) \\ &\rightarrow \{q_3\} \cup \{q_3\} \\ &\rightarrow \{q_3\} \end{aligned}$$



2013 Spring
Convert the NFA $M = (\{q_0, q_1, q_2, q_3\}, \{0, 1\}, \delta, q_0, \{q_2\})$ to its equivalent DFA δ' is given by-

$Q \times \Sigma$	0	1
q_0	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\{q_2\}$	q_1
q_2	$\{q_3\}$	$\{q_3\}$
q_3	$\emptyset$	$\{q_2\}$

Let δ' be the transition function of DFA being formed.

$$\delta'(\{q_0\}, 0) = \delta(q_0, 0) \rightarrow \{q_0, q_1\} \text{ (New state)} \quad (i)$$

$$\delta'(\{q_0\}, 1) \rightarrow \delta(q_0, 1) \rightarrow \{q_0\}$$

$$\delta'(\{q_0, q_1\}, 0) \rightarrow \delta(q_0, 0) \cup \delta(q_1, 0)$$

$$\{q_0, q_1\} \cup \{q_2\}$$

$$\{q_0, q_1, q_2\} \text{ New state} \quad (ii)$$

$$\delta'(\{q_0, q_1\}, 1) \rightarrow \delta(q_0, 1) \cup \delta(q_1, 1)$$

$$\{q_0\} \cup \{q_1\}$$

$$\{q_0, q_1\}$$

$$\delta'(\{q_0, q_1, q_2\}, 0) \rightarrow \delta(q_0, 0) \cup \delta(q_1, 0) \cup \delta(q_2, 0)$$

$$\{q_0, q_1\} \cup \{q_2\} \cup \{q_3\}$$

$$\{q_0, q_1, q_2, q_3\} \text{ New state} \quad (iii)$$

$$\delta'(\{q_0, q_1, q_2\}, 1) \rightarrow \delta(q_0, 1) \cup \delta(q_1, 1) \cup \delta(q_2, 1)$$

$$\{q_0\} \cup \{q_1\} \cup \{q_3\}$$

$$\Rightarrow \{q_0, q_1, q_3\} \text{ New state} \quad (iv)$$

$$\delta'(\{q_0, q_1, q_2, q_3\}, 0) \rightarrow \delta(q_0, 0) \cup \delta(q_1, 0) \cup \delta(q_2, 0) \cup \delta(q_3, 0)$$

$$= \{q_0, q_1, q_2, q_3\}$$

$$\delta'(\{q_0, q_1, q_2, q_3\}, 1) \rightarrow \delta(q_0, 1) \cup \delta(q_1, 1) \cup \delta(q_2, 1) \cup \delta(q_3, 1)$$

$$= \{q_0\} \cup \{q_1\} \cup \{q_3\} \cup \{q_2\}$$

$$\{q_0, q_1, q_2, q_3\}$$

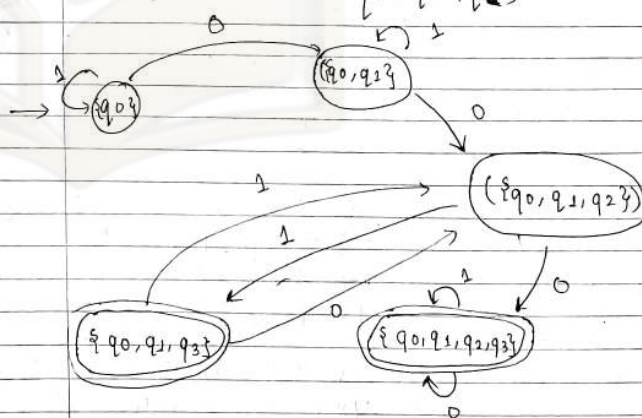
$$\delta'(\{q_0, q_1, q_3\}, 0) \rightarrow \delta(q_0, 0) \cup \delta(q_1, 0) \cup \delta(q_3, 0)$$

$$= \{q_0, q_1, q_2, \emptyset\}$$

$$= \{q_0, q_1, q_2\}$$

$$\delta'(\{q_0, q_1, q_3\}, 1) \rightarrow \delta(q_0, 1) \cup \delta(q_1, 1) \cup \delta(q_3, 1)$$

$$= \{q_0, q_1, q_2\}$$



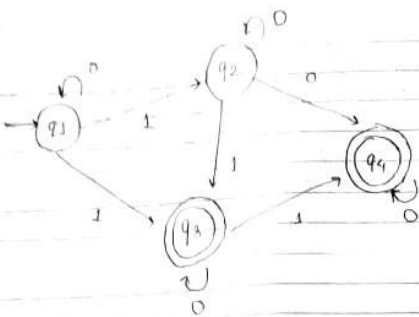


Fig. 3 NFA

All possible sets are: $\{\emptyset\}, \{q_1\}, \{q_2\}, \{q_3\}, \{q_4\}$
 $\{q_1, q_2\}, \{q_1, q_3\}, \{q_1, q_4\}, \{q_2, q_3\}, \{q_2, q_4\}$
 $\{q_3, q_4\}, \{q_1, q_2, q_3\}, \{q_1, q_2, q_4\}, \{q_1, q_3, q_4\}, \{q_2, q_3, q_4\}$

	0	1
q_1	$\{q_1\}$	$\{q_1, q_3\}$
q_2	$\{q_2, q_4\}$	$\{q_3\}$
q_3	$\{q_3\}$	$\{q_4\}$
q_4	$\{q_4\}$	$\{\emptyset\}$

Let δ' be the transition function of DFA being formed

$$\begin{aligned} \delta'(\{q_1\}, 0) &= \delta(q_1, 0) \rightarrow \{q_1\} \\ \delta'(\{q_1\}, 1) &= \delta(q_1, 1) \rightarrow \{q_1, q_3\} \rightarrow \text{New state} \\ \delta'(\{q_2, q_3\}, 0) &= \delta(q_2, 0) \cup \delta(q_3, 0) \\ &= \{q_2, q_4\} \cup \{q_3\} \rightarrow \text{New state} \end{aligned}$$

$$\begin{aligned} \delta'(\{q_2, q_3\}, 1) &= \delta(q_2, 1) \cup \delta(q_3, 1) \\ &= \{q_4\} \cup \{q_4\} \rightarrow \text{New state} \end{aligned}$$

$$\begin{aligned} \delta'(\{q_2, q_3, q_4\}, 0) &= \delta(q_2, 0) \cup \delta(q_3, 0) \cup \delta(q_4, 0) \\ &= \{q_2, q_4\} \cup \{q_3\} \cup \{q_4\} \\ &= \{q_2, q_3, q_4\} \end{aligned}$$

$$\begin{aligned} \delta'(\{q_2, q_3, q_4\}, 1) &= \delta(q_2, 1) \cup \delta(q_3, 1) \cup \delta(q_4, 1) \\ &= \{q_4\} \cup \{q_4\} \cup \{\emptyset\} \\ &= \{q_4\} \end{aligned}$$

$$\begin{aligned} \delta'(\{q_3, q_4\}, 0) &= \delta(q_3, 0) \cup \delta(q_4, 0) \\ &= \{q_3, q_4\} \end{aligned}$$

$$\begin{aligned} \delta'(\{q_3, q_4\}, 1) &= \delta(q_3, 1) \cup \delta(q_4, 1) \\ &= \{q_4\} \cup \{\emptyset\} \\ &= \{q_4\} \end{aligned}$$

~~q_1~~

~~q_2, q_3~~

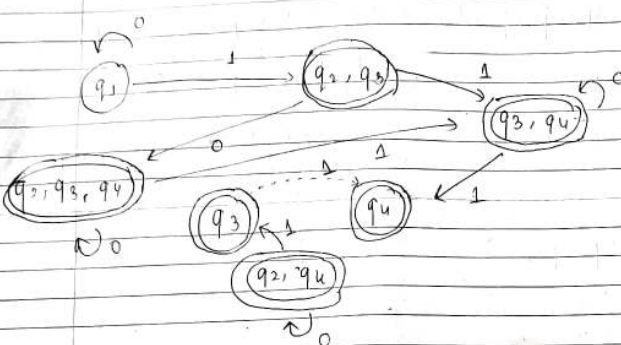
~~q_4~~

~~q_3, q_4~~

~~q_2, q_3, q_4~~

$$\begin{aligned} \delta'(\{q_2, q_4\}, 0) &= \delta(q_2, 0) \cup \delta(q_4, 0) \\ &= \{q_2, q_4\} \cup \{q_4\} \\ &= \{q_2, q_4\} \end{aligned}$$

$$\begin{aligned} \delta'(\{q_2, q_4\}, 1) &= \delta(q_2, 1) \cup \delta(q_4, 1) \\ &= \{q_4\} \cup \{\emptyset\} \\ &= \{q_4\} \end{aligned}$$



Since $\{q_3\}$ and $\{q_2, q_4\}$ are not reachable from start state, so they are discarded and it gives

Epsilon-NFA (E-NFA) :

- A FA that changes its state by consuming empty string (ϵ) i.e. without consuming any valid input is called epsilon-NFA.

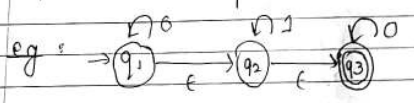


Fig 1:- E-NFA

Epsilon closure

- ϵ -closure(q_1) = $\{q_1, q_2, q_3\}$ → new
- ϵ -closure(q_2) = $\{q_2, q_3\}$ → new
- ϵ -closure(q_3) = $\{q_3\}$ → old.

Conversion of E-NFA to DFA (elimination of empty moves) :-

- To eliminate empty moves from a given E-NFA, we need to calculate epsilon-closure of every state of the given E-NFA.
- epsilon-closure of a particular state is defined as the continuous path that is labelled with ϵ .
- epsilon-closure of every state gives states in the form of old states and new states.
- Then, we identify the transitions for all the new states.
- Transition is calculated as epsilon-closure of the new state.
- After calculating transitions for all the new states, we draw a transition diagram, which is the required FA without empty moves.

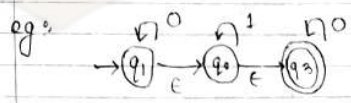


Fig 1:- E-NFA

Here,

$$\begin{aligned} \epsilon\text{-closure}(q_1) &\rightarrow \{q_1, q_2, q_3\} \\ \epsilon\text{-closure}(q_2) &\rightarrow \{q_2, q_3\} \\ \epsilon\text{-closure}(q_3) &\rightarrow \{q_3\} \end{aligned}$$

Let δ' be the transition function of the FA being formed.

Here, $\{q_1, q_2, q_3\}$ and $\{q_2, q_3\}$ are new states.

$$\begin{aligned} \delta'(\{q_1, q_2, q_3\}, 0) &\rightarrow \epsilon\text{-closure}(\delta(\{q_1, q_2, q_3\}, 0)) \\ &\rightarrow \epsilon\text{-closure}(\delta(q_1, 0) \cup \delta(q_2, 0) \cup \delta(q_3, 0)) \\ &\rightarrow \epsilon\text{-closure}(q_1 \cup \phi \cup q_3) \\ &\rightarrow \epsilon\text{-closure}(q_1 \cup q_3) \\ &\rightarrow \epsilon\text{-closure}(q_1) \cup \epsilon\text{-closure}(q_3) \\ &\rightarrow \{q_1, q_2, q_3\} \cup \{q_3\} \\ &\rightarrow \{q_1, q_2, q_3\} \end{aligned}$$

$$\begin{aligned} \delta'(\{q_1, q_2, q_3\}, 1) &\rightarrow \epsilon\text{-closure}(\delta(\{q_1, q_2, q_3\}, 1)) \\ &\rightarrow \epsilon\text{-closure}(\delta(q_1, 1) \cup \delta(q_2, 1) \cup \delta(q_3, 1)) \\ &\rightarrow \epsilon\text{-closure}(\phi \cup q_2 \cup \phi) \\ &\rightarrow \epsilon\text{-closure}(q_2) \\ &\rightarrow \{q_2, q_3\} \end{aligned}$$

$$\begin{aligned} \delta'(\{q_2, q_3\}, 0) &\rightarrow \epsilon\text{-closure}(\delta(\{q_2, q_3\}, 0)) \\ &\rightarrow \epsilon\text{-closure}(\delta(q_2, 0) \cup \delta(q_3, 0)) \\ &\rightarrow \epsilon\text{-closure}(\phi \cup q_3) \\ &\rightarrow \epsilon\text{-closure}(q_3) \\ &\rightarrow \{q_3\} \end{aligned}$$

$$\begin{aligned} \delta'(\{q_2, q_3\}, 1) &\rightarrow \epsilon\text{-closure}(\delta(\{q_2, q_3\}, 1)) \\ &\rightarrow \epsilon\text{-closure}(\delta(q_2, 1) \cup \delta(q_3, 1)) \\ &\rightarrow \epsilon\text{-closure}(q_2 \cup \phi) \\ &\rightarrow \{q_2, q_3\} \end{aligned}$$

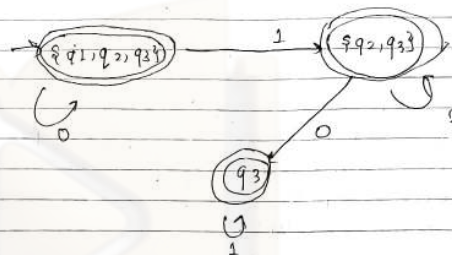
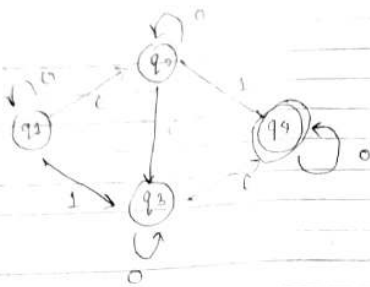


Fig 3 DFA



Epsilon closure

$$\begin{aligned} \epsilon\text{-closure}(q_1) &= \{q_1, q_3\} \\ \epsilon\text{-closure}(q_2) &= \{q_2, q_4\} \\ \epsilon\text{-closure}(q_3) &= \{q_3, q_4\} \\ \epsilon\text{-closure}(q_4) &= \{q_4\} \end{aligned}$$

Let δ' be the transition function of the FA being formed.

$$\begin{aligned} \delta'(\{q_1, q_2, q_3, q_4\}, 0) &\rightarrow \epsilon\text{-closure}(\delta(\{q_1, q_2, q_3, q_4\}, 0)) \\ &= \epsilon\text{-closure}(\delta(q_1, 0) \cup \delta(q_2, 0) \cup \delta(q_3, 0) \cup \delta(q_4, 0)) \\ &= \epsilon\text{-closure}(q_2 \cup q_3 \cup q_4) \\ &= \{q_2, q_3, q_4\} \end{aligned}$$

$$\delta'(\{q_1, q_2, q_3, q_4\}, 1) \rightarrow \epsilon\text{-closure}(\delta(q_1, 1) \cup \delta(q_2, 1) \cup \delta(q_3, 1) \cup \delta(q_4, 1))$$

$$= \epsilon\text{-closure}(q_3 \cup q_4 \cup \phi \cup \phi)$$

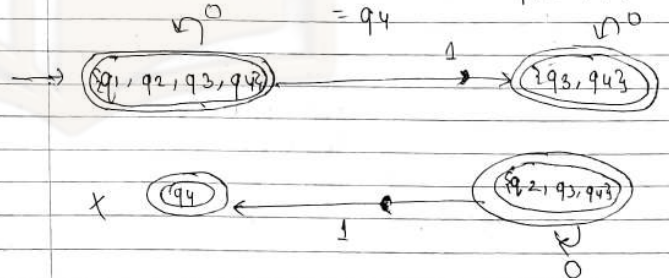
$\{q_3, q_4\}$ now

$$\begin{aligned} \delta'(\{q_3, q_4\}, 0) &\rightarrow \epsilon\text{-closure}(\delta(q_3, 0) \cup \delta(q_4, 0)) \\ &= \epsilon\text{-closure}(q_3 \cup q_4) \\ &= \{q_3, q_4\} \end{aligned}$$

$$\begin{aligned} \delta'(\{q_3, q_4\}, 1) &\rightarrow \epsilon\text{-closure}(\delta(q_3, 1) \cup \delta(q_4, 1)) \\ &= \epsilon\text{-closure}(\phi \cup \phi) \\ &= \phi \end{aligned}$$

$$\begin{aligned} \delta'(\{q_2, q_3, q_4\}, 0) &\rightarrow \epsilon\text{-closure}(\delta(q_2, 0) \cup \delta(q_3, 0) \cup \delta(q_4, 0)) \\ &= \epsilon\text{-closure}(q_3 \cup q_3 \cup q_4) \\ &= \{q_2, q_3, q_4\} \end{aligned}$$

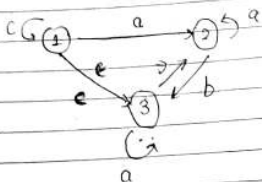
$$\begin{aligned} \delta'(\{q_2, q_3, q_4\}, 1) &\rightarrow \epsilon\text{-closure}(\delta(q_2, 1) \cup \delta(q_3, 1) \cup \delta(q_4, 1)) \\ &= \epsilon\text{-closure}(q_4 \cup \phi \cup \phi) \\ &= q_4 \end{aligned}$$



2015 spring

Eliminate the epsilon transition from the ϵ -NFA
 $N = (\{1, 2, 3\}, \{a, b, c\}, \delta, 1, \{3\})$ where the
 transition function is-

state	a	b	c	ϵ
1	2	ϕ	$\{1, 3\}$	3
2	2	3	ϕ	
3	3	ϕ	ϕ	2



ϵ -closure $\rightarrow \{1\} \rightarrow \{1, 3, 2\}$
 $(2) \rightarrow \{2\}$
 $(3) \rightarrow \{2, 3\}$

Let δ' be the transition fxn of the FA being formed.

$$\begin{aligned} \delta'(\{1, 2, 3\}, a) &\rightarrow \epsilon\text{-closure} \delta(1, a) \cup \delta(2, a) \cup \delta(3, a) \\ &= \epsilon\text{-closure} \{2\} \cup \{2\} \cup \{3\} \\ &= \epsilon\text{-closure} \{2\} \cup \{3\} \\ &= \{2, 3\} \end{aligned}$$

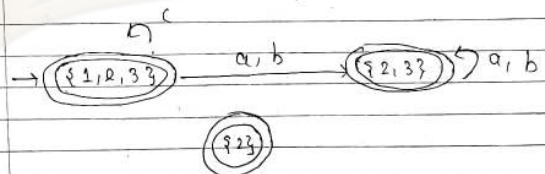
$$\begin{aligned} \delta'(\{1, 2, 3\}, b) &\rightarrow \epsilon\text{-closure} \delta(1, b) \cup \delta(2, b) \cup \delta(3, b) \\ &= \epsilon\text{-closure} \{\phi\} \cup \{3\} \cup \phi \\ &= \{2, 3\} \end{aligned}$$

$$\begin{aligned} \delta'(\{1, 2, 3\}, c) &\rightarrow \epsilon\text{-closure} \delta(1, c) \cup \delta(2, c) \cup \delta(3, c) \\ &= \epsilon\text{-closure} \{1, 3\} \cup \phi \cup \phi \\ &= \epsilon\text{-closure} \{1\} \cup \epsilon\text{-closure} \{3\} \\ &= \{1, 2, 3\} \end{aligned}$$

$$\begin{aligned} \delta'(\{2, 3\}, a) &\rightarrow \epsilon\text{-closure} \delta(2, a) \cup \delta(3, a) \\ &= \{2, 3\} \end{aligned}$$

$$\begin{aligned} \delta'(\{2, 3\}, b) &\rightarrow \epsilon\text{-closure} \{\delta(2, b), \delta(3, b)\} \\ &= \epsilon\text{-closure} \{\phi\} \cup \{3\} \\ &= \{2, 3\} \end{aligned}$$

$$\begin{aligned} \delta'(\{2, 3\}, c) &\rightarrow \epsilon\text{-closure} (\delta(2, c), \delta(3, c)) \\ &\rightarrow \epsilon\text{-closure} \{\phi\} \cup \phi \\ &= \phi \end{aligned}$$



Regular Expressions

An expression used to generate string for FA is called regular expression. It is called language generation. A regular expression is recursively defines as:

1. Empty string (ϵ), empty state ($\emptyset$) and symbols of input alphabet are regular expression.
2. If R_1 and R_2 are two regular expression the union of R_1 and R_2 denoted by $R_1 + R_2$ is also a regular expression.
3. If R_1 and R_2 are two regular expression then concatenation of R_1 & R_2 denoted by $R_1 R_2$ is also regular expression.
4. If R is a regular expression then closure of R denoted by R^* is also regular expression.
5. If R is regular expression, then (R) is also regular expression.

eg

$$0 + (0+1)^* + 1 + 1$$

eg

$$\begin{aligned} (0+1)^* &= \{ \epsilon, 0, 1, 00, 01, 101, 1101, \dots \} \\ (0^* + 1^*) &= \{ \epsilon, 0, 1, 00, 11, 000, 111, \dots \} \\ (01)^* &= \{ \epsilon, 01, 0101, 010101, \dots \} \\ 0^* 1^* &= \{ \epsilon, 0, 1, 01, 001, 011, 0011, \dots \} \\ 1 + 1^* &= \{ 1, 11, 111, 1111, \dots \} \end{aligned}$$

Self String

Design a FA that accepts a set of string such that every string end in 00, over alphabet $\{0, 1\}$.

$$RE = (0+1)^* 00$$

Regular expression to FA

- For every regular expression there exists a path from initial state to final state such that the path is labelled with regular expression.

Unions $R = 0+1$

$$RE = 0+1$$

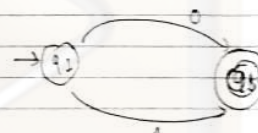
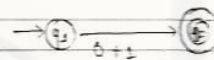


Fig 1 FA

Concatenation

$$RE = 0.1$$

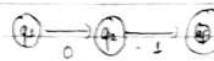
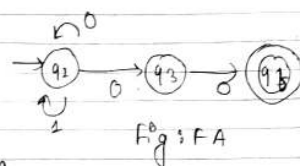
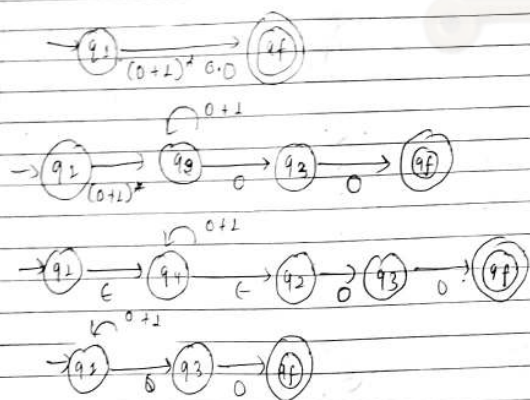
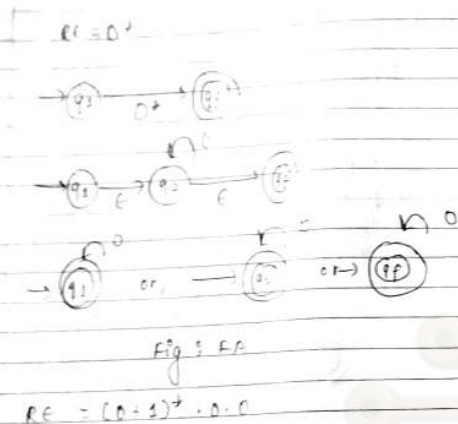


Fig 2 FA



2014 Spring

$$RE = 01^* + 1$$

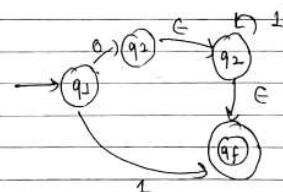
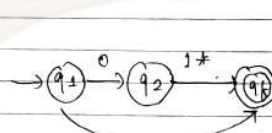
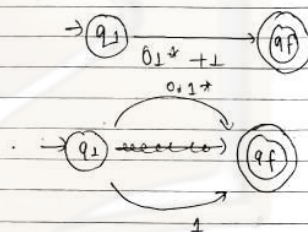
2010 Spring

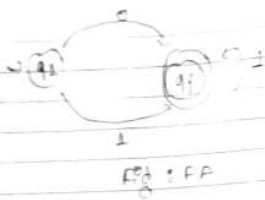
$$RE = a(a+b)^*bb$$

2011 Fall

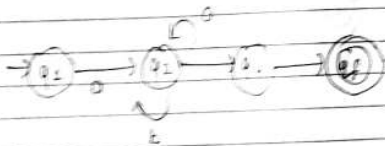
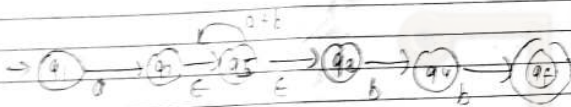
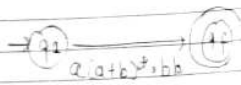
$$RE = (a + a(b+aa)^*b)^*a(b+aa)^*a$$

$$RE : 01^* + 1$$





$$R = a(a+ab)^*bb$$



2019

$$R = (a+ab(b+aa)^*b)^*a(b+aa)^*a$$

FA is Regular expression:-

Aiden's theorem:-

Let P and Q be two regular expressions such that P does not contain empty string (ϵ). Then, a regular expression R of the form,

$$R = Q + RP$$

here, a unique solution

$$R = QP^*$$

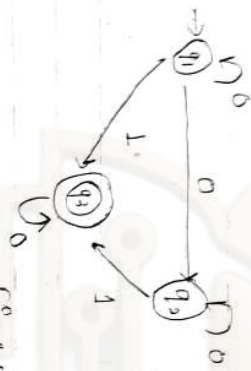


Figure 1.1

$$q_1 = q_0 \cdot 0 + \epsilon \quad \text{--- (i)}$$

$$q_0 = q_1 \cdot 0 + q_2 \cdot 0 \quad \text{--- (ii)}$$

$$q_2 = q_1 \cdot 1 + q_2 \cdot 1 + q_3 \cdot 0 \quad \text{--- (iii)}$$

From eqⁿ ①,

$$q_1 = q_1 \cdot 0 + \epsilon$$

$$R = R \cdot P + \epsilon$$

Using theorem, we get

$$R = 0^* P^+$$

i.e. $q_1 = \epsilon \cdot 0^*$

$$q_1 = 0^* \quad \text{--- ④}$$

From equation ② & ④, we get

$$q_2 = \underbrace{0^* 0}_R + \underbrace{q_2 \cdot 0}_R P$$

Using Arden's theorem we get

$$q_2 = 0^* 0 0^* \quad \text{--- ⑤}$$

Using

From equations ③, ④ & ⑤, we get,

$$\underbrace{q_2}_{R} = \underbrace{0^* 1}_{R} + \underbrace{0^* 0 \cdot 0^* 1}_{R} + \underbrace{q_3 \cdot 0}_{P}$$

Using Arden's theorem we get,

$$R = 0^* P^+$$

$$q_3 = (0^* 1 + 0^* 0 0^* 1) 0^*$$

$$q_3 = 0^* 1 0^* + 0^* 0 0^* 1 0^*$$

Since q_3 is the final state, required regular expression is $0^* 1 0^* + 0^* 0 0^* 1 0^*$

#

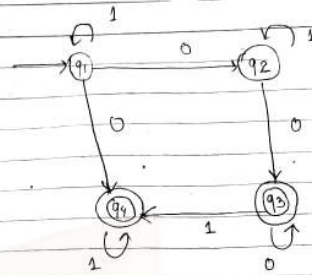


Fig: FA

$$q_1 = q_1 \cdot 1 + \epsilon \quad \text{--- ①}$$

$$q_2 = q_1 \cdot 0 + q_2 \cdot 1 \quad \text{--- ②}$$

$$q_3 = q_2 \cdot 0 + q_3 \cdot 0 \quad \text{--- ③}$$

$$q_4 = q_1 \cdot 0 + q_3 \cdot 1 + q_4 \cdot 1 \quad \text{--- ④}$$

From eqⁿ ①

$$q_1 = q_1 \cdot 1 + \epsilon$$

$$R = R \cdot P + \epsilon$$

Using theorem, we get

$$R = 1^* P^+$$

$$\text{i.e. } q_1 = \epsilon \cdot 1^*$$

$$q_1 = 1^* \quad \text{--- ⑤}$$

From eqⁿ ② and ⑤ we get

$$\underbrace{q_2}_{R} = \underbrace{1^* 0}_{R} + \underbrace{q_2 \cdot 1}_{R \cdot P}$$

Using Arden's theorem:

$$R = QP^+$$

$$q_2 = 1^+ 0 1^+ \quad \text{--- (6)}$$

From eqn (2), (5), (6) we get

$$\underline{q_2} = \underline{1^+ 0 1^+ 0} + \underline{q_2} \cdot \underline{0} \quad \text{--- (7)}$$

From eqn (2), (5), (6), (7)

Using Arden's theorem

$$q_3 = 1^+ 0 1^+ 0 1^+ 0 1^+ \quad \text{--- (7)}$$

Using From eqn (4), (5), (6), (7)

$$\underline{q_4} = \underline{1^+ 0} + \underline{1^+ 0 1^+ 0 0^+ 1} + \underline{q_4} \cdot \underline{1} \quad \text{--- (8)}$$

Using Arden's theorem

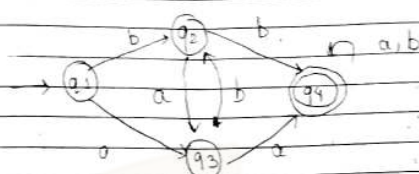
$$q_4 = (1^+ 0 + 1^+ 0 1^+ 0 0^+ 1)^+ 1^+$$

$$q_4 = 1^+ 0 1^+ + 1^+ 0 1^+ 0 0^+ 1 1^+$$

Since q_3 and q_4 are final state, the required reg. expression is: $1^+ 0 1^+ 0 0^+ + 1^+ 0 1^+ + 1^+ 0 1^+ 0 0^+ 1 1^+$

Sol: Fall

Find the regular expression for the following transition diagram



$$q_1 = \epsilon \quad \text{--- (1)}$$

$$q_2 = q_1 \cdot b + q_2 \cdot a \quad \text{--- (2)}$$

$$q_3 = q_1 \cdot b + q_2 \cdot a + q_3 \cdot a \quad \text{--- (3)}$$

$$q_4 = q_3 \cdot a + q_4 \cdot b + q_4 \cdot a, b \quad \text{--- (4)}$$

$$q_2 = q_1 \cdot b + q_2 \cdot a$$

$$q_2 = \epsilon \cdot b + q_2 \cdot a$$

$$\underline{q_2} = \underline{b} + \underline{q_2} \cdot \underline{a}$$

$$q_2 = b + q_2 a \cdot b + q_2 \cdot a b$$

$$q_2 = b + \epsilon \cdot a b + q_2 \cdot a b$$

$$\underline{q_2} = \underline{b} + \underline{a b} + \underline{q_2} \cdot \underline{a b}$$

$$R = QP^+$$

$$q_2 = (b + ab) (ab)^+$$

$$q_1 = b (ab)^+ + ab (ab)^+ \quad \text{--- (5)}$$

From (iii), (i) & (v)

$$q_3 = \epsilon \cdot a + (b(ab)^* + ab(ab)^*) \cdot a$$

$$q_3 = a + b(ab)^*a + ab(ab)^*a$$

From (iii), (i) & (v)

$$q_3 = \underbrace{a}_{R} + \underbrace{b(ab)^*a}_{Q} + \underbrace{ab(ab)^*a}_{R \cdot P}$$

$$R = QP^*$$

$$q_3 = (a + ba)(ba)^*$$

$$q_3 = a(ba)^* + ba(ba)^* \quad \text{--- (vi)}$$

From (iv), (i) & (vi)

$$q_4 = \underbrace{(a+b)}_R \underbrace{q_4}_P + \underbrace{a(b(ba)^* + ba(ba)^*) + b(b(ab)^* + (ab)^*)}_Q$$

$$q_4 = QP^*$$

$$= (ab(ba)^* + aba(ba)^* + bba(ab)^* + b(ab)^*) \cdot (a+b)^*$$

$$q_4 = ab(ba)^*(a+b)^* + aba(ba)^*(a+b)^* + bba(ab)^*(a+b)^* + b(ab)^*(a+b)^*$$

This is not correct.

* Closure properties of regular sets or language:-

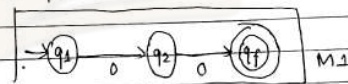
The class of language of FA is closed under:

- (i) Union
- (ii) Concatenation
- (iii) Kleene Closure

(i) Union:

Let R_1 and R_2 be two regular expressions. Then, we have to show that the Union of R_1 and R_2 i.e. $R_1 + R_2$ is also a regular expression. We know that for every expression there exists an equivalent FA. Therefore, to show that $R_1 + R_2$ is regular, we need to design a FA that can process either the strings generated by R_1 or strings generated by R_2 . Let $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ and $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ be the finite automata for R_1 and R_2 respectively. Again, Let $R_1 = 00$ and $R_2 = 11$

Now, FA for R_1 and R_2 will be as:



Let $M = (Q, \Sigma, \delta, q_0, F)$ be a FA such that we need to define its operational characteristics in such a way that it process the strings of R_1 or R_2 .

Its operational characteristics are defined as

$$Q = Q_1 \cup Q_2 \cup q$$

$$\Sigma = \Sigma_1 \cup \Sigma_2$$

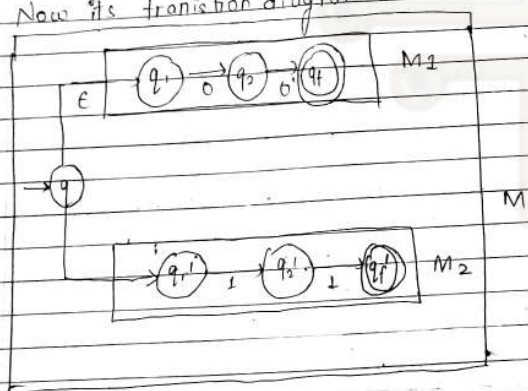
δ consists of

$$\delta_1 \cup \delta_2 \cup \{ \delta(q, \epsilon) \rightarrow q_1, \delta(q, \epsilon) \rightarrow q_2' \}$$

$$q_0 = q$$

$$F = F_1 \cup F_2$$

Now its transition diagram is:



Here, q is the initial state of M . The transition fns $\delta(q, \epsilon) \rightarrow q_1$ and $\delta(q, \epsilon) \rightarrow q_1'$ tells that M can transit to either the initial

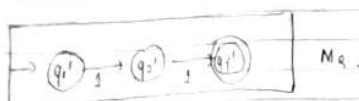
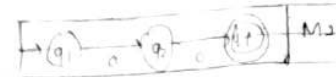
state of M_1 or initial state of M_2 by consuming ϵ non-deterministically. When M uses transition fn $\delta(q, \epsilon) \rightarrow q_1$, it transits to initial state of M_1 , uses transition fn δ_1 and processes all strings generated by R_1 . Similarly, when M uses transition fn $\delta(q, \epsilon) \rightarrow q_1'$, it transits to initial state of M_2 , uses transition fns δ_2 and processes all the strings generated by R_2 i.e. M copies the processing of either M_1 or M_2 . Hence we say that M is such a FA that can process either the strings generated by R_1 or string ϵ generated by R_2 . Therefore, $R_1 + R_2$ is also regular. Thus, the class language of FA is closed under union.

(ii) Concatenations:

Let R_1 and R_2 be two regular expression. Then we have to show that the concatenation of R_1 and R_2 i.e. $R_1 R_2$ is also regular expression. We know that for every expression there exists an equivalent FA. Therefore, to show $R_1 R_2$ is regular, we need to design a FA that process the strings generated by R_1 and R_2 . Let $M_1 = (Q_1, \Sigma_1, \delta_1, q_1, F_1)$ automata for R_1 and $M_2 = (Q_2, \Sigma_2, \delta_2, q_2, F_2)$ be the finite automata for R_1 and R_2 respectively.

Again, let $R_1 = 00$ and $R_2 = 11$

The FA for R_1 and R_2 will be as:



Let $M = (Q, \Sigma, \delta, q_0, F)$ be a FA such that we need to define its operation characteristics in such a way that it processes the strings of R_1 and followed by strings of R_2 .

Its operational characteristics are defined as:

$$Q = Q_1 \cup Q_2 \cup q$$

$$\Sigma = \Sigma_1 \cup \Sigma_2$$

δ consists of

$$\delta_1, \forall \delta \in \delta_1 \delta(q, \epsilon) \rightarrow q_1, \delta(q, \epsilon) \rightarrow q_1$$

$$q_0 = q$$

$$F = F_2$$

Now its transition diagram is as



Here, q is the initial state of M . The transition fn $\delta(q, \epsilon) \rightarrow q_1$ and $\delta(q, \epsilon) \rightarrow q_1'$ tells that M can transit to initial state of M_1 and M_2 respectively. $\delta(q, \epsilon) \rightarrow q_1'$ tells that final state of M_1 consumes ϵ and goes to initial state of M_2 and by using δ_2 it processes R_2 .

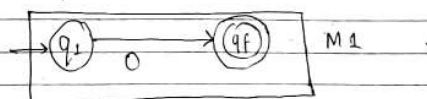
by consuming ϵ . After transiting to initial state of M_2 , it uses transition fn δ_2 and processes all the strings generated by R_2 . Then it uses transition fn $\delta(q, \epsilon) \rightarrow q_1'$ and transits to initial state of M_1 . After that, it uses transition functions δ_1 and processes all the strings generated by R_1 i.e. M can process all the strings of M_1 followed by all the strings of M_2 . Thus, we say that M can process the concatenation of R_1 and R_2 . Hence $R_1 R_2$ is also regular.

3) closure

Let R_1 be the regular expression. Then we have to show that the closure of R_1 is also a regular expression. We know that for every expression there exists an equivalent FA. Therefore, to show that R^* is a regular, we need to design a FA that can process the string generated by R . Let $M = (Q, \Sigma, \delta, q_1, F)$ be the finite automata of R .

Again, $R = \emptyset$

Now, FA for R will be



Let $M = (Q, \Sigma, \delta, q_0, f)$ be FA such that we need to define its operational characteristics in such a way that it can process R .

It's operation characteristics are defined as

$$Q = Q_1 \cup Q_2$$

$$\Sigma = \Sigma_1$$

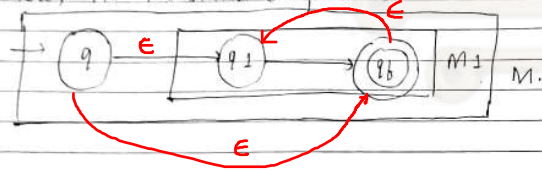
δ consists of

$$\delta_1 \cup \delta_2 \text{ where } \delta_1(q, \epsilon) \rightarrow q_1, \delta_2(q, \epsilon) \rightarrow q_f, \delta_2(q, f, \epsilon) \rightarrow q_2$$

$$q_0 = q$$

$$f = f_1$$

Now, its transition diagram is:



Here, q is the initial state of M . The transition function $\delta(q, \epsilon) \rightarrow q_1$ tells that the initial state M can transit the initial state of M_1 uses transition function δ_1 and process all the strings generated by R . Similarly when M uses transition $\delta_2(q, \epsilon) \rightarrow q_f$ tells that it can process empty string and $\delta(q_f, \epsilon) \rightarrow q_2$ tells that M can process any no. of zero or empty. Since, M is a finite automata that R is also a regular expression.

* Pumping lemma for regular language or regular sets:

Let L be a regular language and $w \in L$ such that $|w| = n$. Let m be the total number of states and $m \leq n$. Then, w can be decomposed into three sub-string x, y and z such that:

- 1) $y \neq \epsilon$
- 2) $|xy| \leq m$
- 3) $xy^i z \in L$ for all $i \geq 0$

Proof:

Let L be regular language and $w \in L$. Let $w = a_1 a_2 \dots a_n$ be the string & $q_1, q_2, \dots, q_m$ be the total no. of states.

Now, according to the pumping lemma, we can decompose into three-sub-string as:

$$x = a_1 a_2 \dots a_i$$

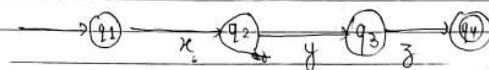
$$y = a_{i+1} a_{i+2} \dots a_j$$

$$z = a_{j+1} a_{j+2} \dots a_n$$

We know that

$$w = xyz \in L$$

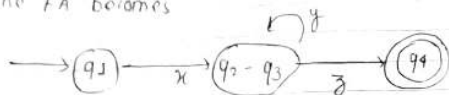
Now, FA becomes P_5 ,



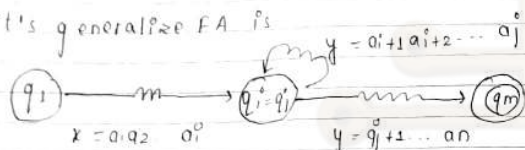
Here, $m = 4$ & $n = 3$

But, $m \leq n$. Therefore, according to the pigeon hole principle any two state must coincide. Let q_2 and q_3 be such states.

Now, the FA becomes



It's generalize FA is



from the above FA, we can write

$$1 \leq i < j \leq m$$

$$1 \leq i < j \leq n$$

$$|xyz| \leq n$$

When

$$i = 0$$

$$x = xz$$

When x is inputted to q_1 , it transit to state $q_i (= q_j)$ and after consuming z it transit to state q_m (final state). Therefore, xz is accepted i.e. $xz \in L$.

When $i = 1$

$$w = xyz \in L$$

When $i = 2$

$$xy^2z \in L$$

similarly,

$$xy^iz \in L \text{ for all } i \geq 0 //$$

⇒ Show that the language $L = \{a^n b^n : n \geq 0\}$ is not regular.

→ Let $L = \{a^n b^n : n \geq 0\}$ is regular and $w = a^p b^p \in L$.

According to pumping lemma, w can be decomposed into three sub-string x, y, z as:

$$x = a^r, y = a^{p-r} b^r, z = a^{p-r} b^p$$

$$\begin{aligned} \text{Now, } xy^2z &= a^{2r} a^{p-r} b^r a^{p-r} b^p \\ &= a^{2r+p-r} a^{p-r} b^r b^p \\ &= a^{p+r} a^{p-r} b^p \end{aligned}$$

Since $r > 0, p+r > p$

ie $a^{p+r} b^p$ is not of the form of $a^n b^n$

ie $xy^2z \notin L$

Hence, L is not regular.

⇒ Show that the language $L = \{1^n : n \geq 0\}$ is not regular.

→ Let $L = \{1^n : n \geq 0\}$ is regular and $w = 1^p \in L$.

According to pumping lemma, w can be decomposed into three sub-string x, y, z as:

$$x = 1^r, y = 1^{p-r}, z = 1^{p-r}$$

$$\begin{aligned} \text{Now, } xy^2z &= 1^r (1^{p-r})^2 1^{p-r} \\ &= 1^{r^2 + 2r^2 + p^2 - r^2 - r^2} \\ &= 1^{p^2 + r^2} \end{aligned}$$

Since, $r^2 > 0, p^2 + r^2 > p^2$

ie $1^{p^2 + r^2}$ is not of the form 1^p

ie $xy^2z \notin L$

Hence, L is not regular.

reverse

⇒ Show that the language $L = \{x x^R : x \in \{0, 1\}^*\}$ is not regular.

→ Let $x = 0^r 1^p$

$x^R = 1^p 0^r$

ie $L = \{0^r 1^p 1^p 0^r : r, p \geq 0\}$ is regular

and $w = 0^p 1^p 0^p 1^p \in L$

According to pumping lemma, w can be decomposed into three sub-string x, y, z as:

$$x = 0^r$$

$$y = 0^{p-2r} 1^p$$

$$z = 0^{p-2r} 1^p 0^p$$

$$\begin{aligned} \text{Now, } xy^2z &= 0^r 0^{2(p-2r)} 1^{2p} 0^{p-2r} 1^p 0^p \\ &= 0^{r+2p-4r+p-2r} 1^{2p+p} 0^p \\ &= 0^{p+1} 1^{3p} 0^p \end{aligned}$$

i.e. $O^{P^4} \perp P^1 P^0 O^P$ is not of the form $O^P \perp P^1 P^0 O^P$
Hence, $\mathcal{L}$ is not regular

i.e. $O^{P^4} \perp P^1 P^0 O^P$ is not of the form $O^P \perp P^1 P^0 O^P$
Hence, $\mathcal{L}$ is not regular

A grammar is defined as:-
 $G = (V, \Sigma, R, S)$

A grammar is defined as:-
 $G = (V, \Sigma, R, S)$

$$G_1 = (V, E, R, S)$$
 Z - set of terminals

R - set of production rules

s = start symbol

eg: for the regular expression $R = a(a^* + b^*)b$,
equivalent grammar is -

Equivalent grammar is -

$$G = (V, \frac{1}{2}, R, S)$$

where,

$$V = \{S, M, A, B\}$$
$$\Sigma = \{a, b\}$$

R consists of

$s \rightarrow a m b$

$$M \rightarrow A|B$$
$$A \rightarrow \epsilon \mid aA$$
$$E \rightarrow E / bB$$

ϵ = start symbol

$w = abbbb$

$$S \rightarrow a M b$$
$$\rightarrow a b b \quad [\cdot M \rightarrow b]$$
$$\rightarrow a b B b \quad [\therefore B \rightarrow b B]$$
$$\rightarrow a b b b b b \quad [: b \rightarrow b b]$$
$$\rightarrow abbbBb \text{ [} \because B \rightarrow bB \text{]}$$
$$\rightarrow abbbh \quad [\because B \rightarrow b]$$

$\rightarrow a b b b b$

Construct an equivalent grammar of the following regular expression.

$$RE = 01^* 010^* \mid 0^* 11(0+1)^*$$
$$10/ = 10/ \times 1/2 \times 10/ \times 1/2 \times 10/ \times 1/2$$
$$= 0.12$$
$$S \rightarrow 01^*010^* \mid 0^*11(0+1)^*$$

↓ use this
production rule

$$\therefore S \rightarrow A/B$$

$A \rightarrow 00010$

$$D \rightarrow 011 \in$$
$$c \rightarrow e \mid 1c$$

$\frac{d}{dt}$	f/g
----------------	-------

$$E \rightarrow E / \cup E / \wedge E$$

#1 Types of Grammar (Chomsky hierarchy of grammar):-

1. Unrestricted grammar (Type 0)
2. Context sensitive grammar (Type 1)
3. Context free grammar (Type 2)
4. Regular grammar (Type 3)

- If no restriction is applied to production rules of a grammar then it is called unrestricted grammar.
- A grammar is said to be context sensitive grammar if its production rules are of the form -
 $w_1 \alpha w_2 \rightarrow w_1 \beta w_2$

Here, $\alpha, \beta \in V$ and w_1, w_2 are called contexts of α and β .

- A grammar is said to be context free grammar if its production rules are of the form -

$$\alpha \rightarrow \beta$$

where, $\alpha \in V$ and $\beta \in (V \cup \Sigma)^*$

- A grammar is said to be regular grammar if its production rules are of the form
 $\text{non-terminal} \rightarrow \text{exactly one terminal}$
or
 $\text{non-terminal} \rightarrow \text{one terminal followed by one non-terminal.}$

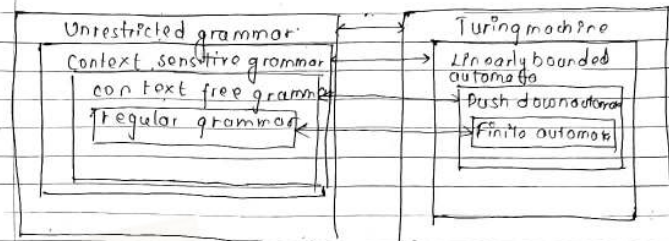


fig: Types of grammar with their respective mathematical models.

Derivations:-

- The sequence of production rules used to generate a string is called derivation.

eg: $u_1, w_2, \dots, w_n$

$$w_1 \xrightarrow{P} w_n$$

- There are two types of derivations

① Leftmost derivation.

② Rightmost derivation

eg:

$$S \rightarrow AB$$

$$A \rightarrow aA|b$$

$$B \rightarrow bB|a$$

$$w = aabbbba$$

Leftmost derivation:-

$S \rightarrow AB$
 $\rightarrow aAB [A \rightarrow aA]$
 $\rightarrow aaAB [A \rightarrow aA]$
 $\rightarrow aabb [A \rightarrow b]$
 $\rightarrow aabbB [B \rightarrow bB]$
 $\rightarrow aabbbB [B \rightarrow bB]$
 $\rightarrow aabbbba [B \rightarrow a]$

Rightmost derivation:-

$S \rightarrow AB$
 $\rightarrow AbB [B \rightarrow bB]$
 $\rightarrow Abbb [B \rightarrow bB]$
 $\rightarrow Abba [B \rightarrow a]$
 $\rightarrow aAbba [A \rightarrow aA]$
 $\rightarrow aaAbba [A \rightarrow aA]$
 $\rightarrow aabbbba [A \rightarrow b]$

$\# S \rightarrow aAs/as/b$
 $A \rightarrow bB/a$
 $B \rightarrow aA/b$
 $w: abbaaqaab$

Leftmost derivation:

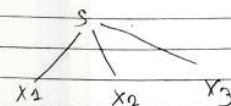
$S \rightarrow aAs/as/b$
 $\rightarrow aAS [A \rightarrow bB]$
 $\rightarrow abbS [A \rightarrow aA] [B \rightarrow bB]$
 $\rightarrow abbaS [B \rightarrow aA] [B \rightarrow bB]$
 $\rightarrow abbaaS [B \rightarrow aA] [B \rightarrow aA]$
 $\rightarrow abbaaaS [B \rightarrow aA]$
 $\rightarrow abbaaaaaS [B \rightarrow aA]$
 $\rightarrow abbaaaaaa [A \rightarrow a]$
 $\rightarrow abbaaaaaab [S \rightarrow b]$

Rightmost derivation:

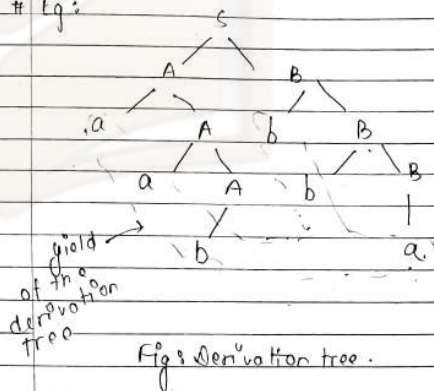
$\rightarrow aAS$
 $\rightarrow aa [S \rightarrow as]$
 $\rightarrow aas [S \rightarrow as]$
 $\rightarrow aaas [S \rightarrow as]$
 $\rightarrow aaaaa [S \rightarrow ab]$
 $\rightarrow aaaaaa [S \rightarrow as]$
 $\rightarrow aaaaaaa [S \rightarrow as]$
 $\rightarrow aaaaaaab [S \rightarrow b]$
 $\rightarrow abbaaaaaab [A \rightarrow bB]$
 $\rightarrow abbaaaaaab [B \rightarrow aA]$
 $\rightarrow abbaaaaaab [A \rightarrow a]$

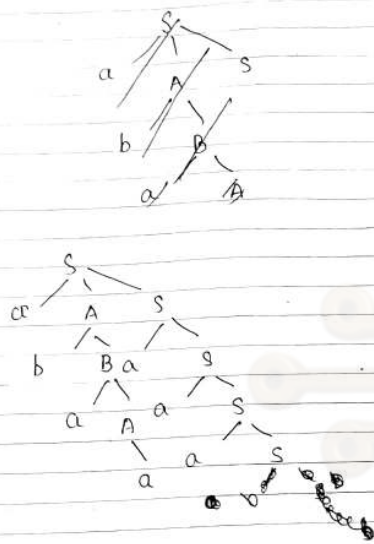
Derivation tree / production trees:

- It is the hierarchical representation of the derivation.
 - The start symbol is the root node of the derivation tree and the string in the right side of start symbol is divided into multiple number of branches.
 - This process is continued at every level of the tree.
- eg: $S \rightarrow x_1 x_2 x_3$ be the production rule, then its derivation tree is -



Eg:





Ambiguity in CFG:

A CFG is said to be ambiguous if there exists more than one leftmost derivation or more than one rightmost derivation for the same string.

eg:

$S \rightarrow aAs/b$	$w = abab$	LMD:-
$A \rightarrow bB/ba$	LMD:-	$S \rightarrow aAs$
$B \rightarrow a$		$\rightarrow abas [A \rightarrow ba]$
is an ambiguous CFG	$S \rightarrow aAs$	$\rightarrow abab [B \rightarrow a]$
	$\rightarrow abBS [A \rightarrow bB]$	
	$\rightarrow abas [B \rightarrow a]$	
	$\rightarrow abab [S \rightarrow b]$	

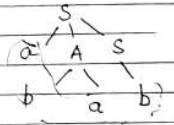
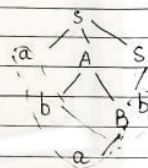


Fig: Derivation tree

Fig: Derivation tree

Show that the following CFG is ambiguous.

$$E \rightarrow E + E \mid E * E \mid a \mid b$$

$$w = a + b * a * b$$

$$\rightarrow E + E * E * b (E \rightarrow b)$$

$$E \rightarrow E + E$$

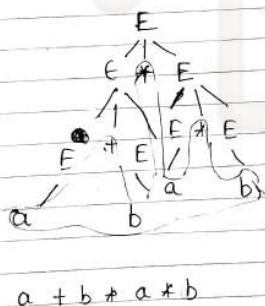
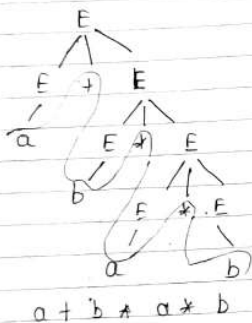
$$\rightarrow E + E * a * b (E \rightarrow a)$$

$$\rightarrow E + E * E (E \rightarrow E * E) \rightarrow E + b * a * b (E \rightarrow b)$$

$$\rightarrow E + E * E * E (E \rightarrow E * E) \rightarrow a + b * a * b (E \rightarrow a)$$

$\rightarrow E + E \quad [E \rightarrow E + E]$
 $\rightarrow a + E \quad [E \rightarrow a]$
 $\rightarrow a + E * E \quad [E \rightarrow E * E]$
 $\rightarrow a + b * E \quad [E \rightarrow b]$
 $\rightarrow a + b * E * E \quad [E \rightarrow E * E]$
 $\rightarrow a + b * a * E \quad [E \rightarrow a]$
 $\rightarrow a + b * a * b \quad [E \rightarrow b]$

* MD
 $\rightarrow E * E$
 $\rightarrow E + E * E \quad [E \rightarrow E + E]$
 $\rightarrow a + E * E \quad [E \rightarrow a]$
 $\rightarrow a + b * E \quad [E \rightarrow b]$
 $\rightarrow a + b * E * E \quad [E \rightarrow E * E]$
 $\rightarrow a + b * a * E \quad [E \rightarrow a]$
 $\rightarrow a + b * a * b \quad [E \rightarrow b]$



Since, there exists more than one CMD, so, CFG is ambiguous.

2017 Fall

check ambiguity of the grammar

$S \rightarrow aB | bA$

$A \rightarrow aS | bAa$

$B \rightarrow aBB | bS | b$

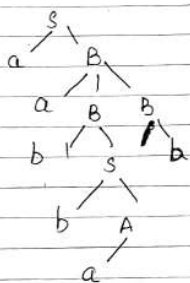
$S \rightarrow aB$
 $\rightarrow a / aBB \quad [B \rightarrow aBB]$
 $\rightarrow a / aBbs \quad [B \rightarrow bS]$
 $\rightarrow a / aab$
 $\rightarrow a / aabb \quad [B \rightarrow aBB]$
 $\rightarrow a / aabbs \quad [B \rightarrow bS]$
 $\rightarrow a / aababb \quad [B \rightarrow b]$
 $\rightarrow a / aababB \quad [B \rightarrow b]$
 $\rightarrow a / aababbb \quad [B \rightarrow b]$

$S \rightarrow aB$
 $\rightarrow a / aBB \quad [B \rightarrow aBB]$
 $\rightarrow a / aBB \quad [B \rightarrow b]$
 $\rightarrow a / aBbs \quad [B \rightarrow bS]$
 $\rightarrow a / aabbbA \quad [S \rightarrow bA]$
 $\rightarrow a / aabbbba \quad [A \rightarrow a]$

$S \rightarrow aB$
 $\rightarrow a / aBB \quad [a \rightarrow aBB]$
 $\rightarrow a / aa$

$S \rightarrow aB$
 $\rightarrow a / aBB \quad [B \rightarrow aBB]$
 $\rightarrow a / aabS \quad [B \rightarrow bS]$
 $\rightarrow a / aabbaB \quad [S \rightarrow bA]$
 $\rightarrow a / aabbb \quad [A \rightarrow bA]$

LMD
 $S \rightarrow aB$ [$S \rightarrow aB$]
 $\rightarrow aBB$ [$B \rightarrow aBB$]
 $\rightarrow aabsB$ [$B \rightarrow Bs$]
 $\rightarrow aabbAB$ [$B \rightarrow bA$]
 $\rightarrow aabbab$ [$A \rightarrow a$]
 $\rightarrow aabbab$ [$B \rightarrow b$]



aabbab

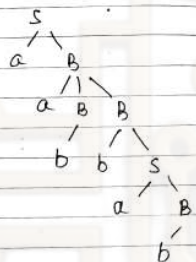
Since, there exists more than one LMD, so grammar is ambiguous.

Normal form of CFG

- If certain restriction is applied to the production rules of CFG then it is said to be in normal form.
- There are two types of normal forms -

- (i) Chomsky Normal form (CNF)
- (ii) Greibach normal form (GNF)

LMD
 $S \rightarrow aB$
 $\rightarrow aBB$ [$B \rightarrow aBB$]
 $\rightarrow aabb$ [$B \rightarrow b$]
 $\rightarrow aabbs$ [$B \rightarrow bs$]
 $\rightarrow aabbAB$ [$S \rightarrow aB$]
 $\rightarrow aabbab$ [$B \rightarrow b$]



aabbab

Chomsky Normal form (CNF):

A CFG is said to be in CNF if its production rules are of the forms-

Non-terminal $\rightarrow$ exactly one terminal

or

Non-terminal $\rightarrow$ exactly two non-terminals.

eg: $S \rightarrow a$, $S \rightarrow AB$, etc.

imp.

Conversion of CFG to CNF:

To convert a given CFG to its equivalent CNF, we need to convert all the production rules of the given CFG to the form-

Non-terminal $\rightarrow$ exactly one terminal

or

Non-terminal $\rightarrow$ exactly two non-terminal

Non-~~terminal~~

To accomplish this task, we use substitution method. First, we replace all terminal symbols in a production rule by non-terminal symbols. But this substitution is possible only when the non-terminals that are substituted generates the respective terminal symbols. Now, add such production rule to the given CFG. eg: $S \rightarrow aA$ be the production rule. To convert it

into its equivalent CNF, we replace 'a' by a nonterminal, say c_1 so that production rule becomes $S \rightarrow c_1 A$. But this is possible only when a production rule $c_1 \rightarrow a$ exists. So, we add $c_1 \rightarrow a$ to the given grammar. We continue this process until all the production rules are converted to CNF.

eg: Convert the following CFG to its equivalent CNF.
 $S \rightarrow aAS | aS$
 $A \rightarrow aBb$
 $B \rightarrow bB | bAa$

1. $S \rightarrow aAS$ gives $S \rightarrow c_1 AS$ where $c_1 \rightarrow a$
- $S \rightarrow c_1 AS$ gives $S \rightarrow c_1 c_2$ where $c_2 \rightarrow AS$
2. $S \rightarrow aS$ gives $S \rightarrow c_1 S$ where $c_1 \rightarrow a$
3. $A \rightarrow aBb$ gives $A \rightarrow c_1 Bc_2$ where $c_1 \rightarrow a$, $c_2 \rightarrow b$
- $A \rightarrow c_1 Bc_2$ gives $A \rightarrow c_1 c_3$ where $c_1 \rightarrow Bc_2$
4. $B \rightarrow bB$ gives $B \rightarrow c_3 B$ where $c_3 \rightarrow b$
5. $B \rightarrow bAa$ gives $B \rightarrow c_3 Ac_4$ where $c_3 \rightarrow b$, $c_4 \rightarrow a$
- $B \rightarrow c_3 Ac_4$ gives $B \rightarrow c_5 c_2$ where $c_5 \rightarrow c_3 A$, $c_2 \rightarrow b$

Let $G' = (V', \Sigma', R', S')$ be the equivalent CNF where,

$$V' = \{S, A, B, c_1, c_2, c_3, c_4, c_5\}$$

$$\Sigma' = \{a, b\}$$

R' consists of -

$$\{S \rightarrow c_1 c_2 | c_1 S, A \rightarrow c_1 c_4, B \rightarrow c_3 B | c_5 c_2, c_2 \rightarrow AS, c_4 \rightarrow Bc_3, c_5 \rightarrow c_3 A, c_1 \rightarrow A, c_3 \rightarrow b\}$$

$$S' = S$$

2016 Spring

Consider the grammar $G = (V, \Sigma, P, S)$ where,

$$V = \{S, A, B\}$$

$$\Sigma = \{a, b\}$$

and the productions P are:

$$S \rightarrow bA | aB$$

$$A \rightarrow bAA | aS | a$$

$$B \rightarrow aBB | bB | b$$

Find an equivalent grammar in CNF.

1. $S \rightarrow bA$ gives $S \rightarrow c_1 A$ where, $c_1 \rightarrow b$
2. $S \rightarrow aB$ gives $S \rightarrow c_2 B$ where, $c_2 \rightarrow a$
3. $A \rightarrow bAA$ gives $A \rightarrow c_3 A$ where $c_3 \rightarrow bA$
- $A \rightarrow c_3 A$ gives $A \rightarrow c_1 A$ where $c_1 \rightarrow bA$
4. $A \rightarrow aS$ gives $A \rightarrow c_4 S$ where $c_4 \rightarrow a$
5. $B \rightarrow aBB$ gives $B \rightarrow c_5 BB$ where $c_5 \rightarrow a$
- $B \rightarrow c_5 BB$ gives $c_5 \rightarrow c_6 B$ where $c_6 \rightarrow aB$
6. $B \rightarrow bB$ gives $B \rightarrow c_7 B$ where $c_7 \rightarrow b$
- $B \rightarrow c_7 B$

Let $G' = (V', \Sigma', P', S')$ be the equivalent CNF where,

$$V' = \{S, A, B, c_1, c_2, c_3, c_4, c_5, c_6, c_7\}$$

$$\Sigma' = \{a, b\}$$

P' consists of

$$\{S \rightarrow c_1 A | c_2 B, A \rightarrow c_1 A | c_4 S | c_3 A | c_3 B | c_3 c_3, c_4 \rightarrow AS, c_5 \rightarrow c_6 B | c_7 B | b, c_1 \rightarrow bA, c_2 \rightarrow aB\}$$

Greibach normal form (GNF):

A CFG is said to be in GNF if its production rules are of the form:-

$$[X \rightarrow a\beta^*]$$

where, $X, \beta \in V$ and $a \in \Sigma$.

Non-terminal $\rightarrow$ exactly one terminal
or

Non-terminal $\rightarrow$ one terminal followed by any number of non-terminals.

eg: $S \rightarrow a, S \rightarrow aA, S \rightarrow aAB$, etc.

Simplification of CFG:

- ① Removal of unit productions.
- ② Removal of null productions.
- ③ Removal of useless productions and symbols.

1. Removal of unit productions:-

A production rule of the form-
Non-terminal $\rightarrow$ non-terminal.

is called unit-production.

eg: $S \rightarrow A, A \rightarrow B$, etc.

Let $X \rightarrow \beta$ be the unit production. To eliminate $X \rightarrow \beta$ from the given CFG, we search for a production rule of the form $\beta \rightarrow w$, where w is the terminal or terminal string (generated directly or indirectly). Then, we replace β in $X \rightarrow \beta$ by w so that a new production rule $X \rightarrow w$ is obtained. We add $X \rightarrow w$ to the given CFG and eliminate $X \rightarrow \beta$. This process is continued until all the unit productions are eliminated.

eg: Simplify the following CFG by eliminating unit productions:

$S \rightarrow AB, A \rightarrow B, B \rightarrow c, C \rightarrow D, D \rightarrow E, E \rightarrow a$.

$\rightarrow$ Soln:

Here, the unit productions are:

$A \rightarrow B$

$B \rightarrow c$

$C \rightarrow D$

$D \rightarrow E$

To eliminate these unit productions, we search such a production rule for B, C, D and E , that generates a terminal or terminal string directly or indirectly.

Here, $E \rightarrow a$ exists.

Now, replace E in $D \rightarrow E$ by 'a' so that a new production rule $D \rightarrow a$ is obtained. We add $D \rightarrow a$

to the given CFG and eliminate $b \rightarrow \epsilon$.

Similarly, we $C \rightarrow a$, $B \rightarrow a$ and $A \rightarrow a$ to the given CFG and eliminate unit productions $C \rightarrow D$, $B \rightarrow C$ and $A \rightarrow B$.

Hence, after eliminating unit productions, the simplified CFG is

$S \rightarrow AB$

$A \rightarrow a$

$B \rightarrow a$

$C \rightarrow a$

$D \rightarrow a$

$\epsilon \rightarrow a$

2. # Removal of null productions:

A production rule of the form
non-terminal $\Rightarrow$ empty string
is called null productions.

eg: $S \rightarrow \epsilon$, $A \rightarrow \epsilon$, etc.

Let $\alpha \rightarrow \epsilon$ be the null production. To eliminate $\alpha \rightarrow \epsilon$ from the given CFG, we search for all the occurrence of α in the production rules of given CFG. Then, we replace α by ϵ in all the production rules where α occurs so that new production rules are obtained. We add these production rules to the given CFG and eliminate $\alpha \rightarrow \epsilon$. This process is continued until all the null productions

are eliminated.

eg simply the following grammar by eliminating null productions.

$S \rightarrow aABs \mid AB$

$A \rightarrow aA \mid \epsilon$

$B \rightarrow bB \mid \epsilon$

$\rightarrow$ Soln:

Here, the null productions are:-

$A \rightarrow \epsilon$

$B \rightarrow \epsilon$

To eliminate $A \rightarrow \epsilon$, we search for the occurrence of A in the production rules of given CFG.

$S \rightarrow aABs$ gives $S \rightarrow aBs$

$S \rightarrow AB$ gives $S \rightarrow B$

$A \rightarrow aA$ gives $A \rightarrow a$

After eliminate $A \rightarrow \epsilon$, the CFG becomes-

$S \rightarrow aABs \mid AB \mid aBs \mid B$

$A \rightarrow aA \mid a$

$B \rightarrow bB \mid \epsilon$

To eliminate $B \rightarrow \epsilon$, we search for occurrences of B in the production rules of new CFG.

Now,

$S \rightarrow aABs$ gives $S \rightarrow aAs$

$S \rightarrow AB$ gives $S \rightarrow A$

$s \rightarrow aBS$ gives $s \rightarrow a$
 $s \rightarrow B$ gives $s \rightarrow \epsilon$
 $s \rightarrow bB$ gives $s \rightarrow b$

After eliminating $B \rightarrow \epsilon$, the CFG becomes-

$s \rightarrow aABs \mid AB \mid Bs \mid B \mid aAs \mid Aa \mid b \mid \epsilon$
 $A \rightarrow aAa$
 $B \rightarrow bBb$

To eliminate $s \rightarrow \epsilon$, we search for occurrences of s in the production rule of new CFG.

Now,

$s \rightarrow aABs$ gives $s \rightarrow aAB$
 $s \rightarrow aAB$ gives $s \rightarrow aA$
 $s \rightarrow aBs$ gives $s \rightarrow aB$
 $s \rightarrow \epsilon$ gives $s \rightarrow a$

After eliminating $s \rightarrow \epsilon$, the CFG becomes

$s \rightarrow aAB \mid aA \mid aB \mid a \mid AB \mid B \mid A \mid b$
 $A \rightarrow aAa$
 $B \rightarrow bBb$

3. Removal of useless productions and symbols :-

To eliminate useless productions and symbols from the given CFG, we first identify all the generating production rules of the given CFG. Generating production rules are the rules that can generate terminal or terminal strings directly or indirectly. The remaining production rules are non-generating production rules, so we eliminate them. Among the generating production rules, some of the production rules may not be ~~red~~ reachable from start symbol. To identify such production rules and symbols, we construct a graph called dependency graph. The non-terminal symbols of the given CFG are represented by vertices of the dependency graph. The vertices that remains disconnected or there exists no direct or indirect path from start symbol, are the symbols that are not reachable from start symbol. Hence, they are also eliminated.

eg: Simplify the following CFG by eliminating useless production and symbols.

$s \rightarrow aAs \mid b$
 $A \rightarrow bA \mid AB \mid a$
 $B \rightarrow bA \mid b \mid a \mid b$
 $D \rightarrow ab$

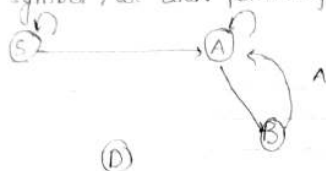
→ soln :

The generating production rules are :-

$S \rightarrow aAB|b$
 $A \rightarrow bA|AB|a$
 $B \rightarrow bA|b$
 $D \rightarrow ab$

The production rule
 $B \rightarrow acb$ is non-generating,
 so it is eliminated

To identify the symbols that are not reachable from start symbol, we draw following dependency graph



here, D remains
 disconnected. So it is
 discarded.

Hence, the simplified
CFG is:

$S \rightarrow aAB|b$
 $A \rightarrow bA|AB|a$
 $B \rightarrow bA|b$

$S \rightarrow aB|bx$
 $A \rightarrow BaD|bsX|a$
 $B \rightarrow aSB|bBX$
 $X \rightarrow sBB|aBX|ad$

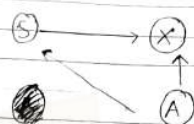
→ The generating production rules are.

$S \rightarrow abx$
 $A \rightarrow bsX|a$
 $B \rightarrow$
 $X \rightarrow ad$

The production rule

$S \rightarrow ab$, $A \rightarrow BaD$, $B \rightarrow aSB|bBX$,
 $X \rightarrow sBB|aBX$ are non-generating
 so, it is eliminated.

To identify the symbols that are not reachable from start symbols, we draw following dependency graph.



There is no direct/indirect path from S to A.

Here, A remains disconnected. So, it is discarded.
 Hence, the simplified
 CFG is:

$S \rightarrow bx$
 $X \rightarrow ad$

Closure properties of context free language:

The class of language of context free grammar.

Is closed under:-

- (i) Union
- (ii) Concatenation
- (iii) Kleene closure.

Union:

Let $G_1 = (V_1, \Sigma, R_1, S_1)$ and $G_2 = (V_2, \Sigma, R_2, S_2)$ be two
 context free languages such that $L_1 \in L(G_1)$ and
 $L_2 \in L(G_2)$. We have to show that the union of
 L_1 and L_2 is also a CFL. To show this,
 we have to design a CFG that can process $L_1 \cup L_2$
 i.e. it can generate either the strings of G_1 or

the strings of Q_2 let $G = (V, \Sigma, R, S)$ be a CFG. Now, we have to define operational characteristics of G in such a way that G can generate all the strings of Q_1 or all the strings of Q_2 . Now, its operational characteristics are defined as:

$V = V_1 \cup V_2 \cup S$
 $\Sigma = \Sigma_1 \cup \Sigma_2$
 R consists of -
 $R_1 \cup R_2 \cup S \rightarrow S_1 S_2$
 $S =$ start symbol

Here, S is the start symbol of G . The production rules $S \rightarrow S_1 S_2$ tell that S can be replaced by either the start symbol of Q_1 or the start symbol of Q_2 . When G uses $S \rightarrow S_1$, S is replaced by S_1 and then uses production rules R_1 and generates all the strings of Q_1 . Similarly, when G uses production rule $S \rightarrow S_2$, then S is replaced by S_2 , uses production rules R_2 and generates all the strings of Q_2 . G can generate either the strings of Q_1 or the string of Q_2 .

Hence, we say that G is a CFG that generate all the strings of Q_1 or Q_2 i.e. it can process $L_1 + L_2$. Yes, $L_1 + L_2$ is also a context free language.

11) Concatenation

Let, $G_1 = (V_1, \Sigma_1, R_1, S_1)$ and $G_2 = (V_2, \Sigma_2, R_2, S_2)$ be two context free languages such that $L_1 \in L(G_1)$ and $L_2 \in L(G_2)$. We have to show the concatenation of L_1 and L_2 i.e. $L_1 L_2$ is also CFL. To show this we have to design a CFG that can process $L_1 L_2$ i.e. it can generate L_1 followed by the strings of G_2 . Let $G = (V, \Sigma, R, S)$ be a CFG.

Now, we have to define operational characteristics of G in such a way that G can generate the strings of G_1 followed by the string of G_2 . Now, its operational characteristics are defined as:

$V = V_1 \cup V_2 \cup S$
 $\Sigma = \Sigma_1 \cup \Sigma_2$
 R consists of -
 $R_1 \cup R_2 \cup S \rightarrow S_1 S_2$
 $S =$ start Symbol

Here, S is the start symbol of G . The production rules $S \rightarrow S_1 S_2$ tells that the S can be replaced by $S_1 S_2$ i.e. by using $S \rightarrow S_1 S_2$, it can generate the strings of G_1 followed by strings of G_2 . First it uses production rules R_1 and generates all strings of G_1 and immediately it uses R_2 and generates all the strings of G_2 i.e. G can process $L_1 L_2$.

(iii) closure

Let $G_1 = (V_1, \Sigma_1, R_1, S_1)$ and $G_2 = (V_2, \Sigma_2, R_2, S_2)$ be two context free grammar and $L_1 \in CFL$, $L_2 \in CFL$. We have to show that the concatenation of closure denoted by L_1^* is also a CFL.

To show that we have to design a CFG that can process L_1^* i.e. it can generate the string of G_2 .

Let $G = (V, \Sigma, R, S)$ be the CFG. Now, we have to define the operational characteristics of G in such a way that G can generate the string of G_2 . Its operation characteristics are:

$$V = V_1 \cup V_2$$

$$\Sigma = \Sigma_1$$

R consists of

$$R_1 \cup R_2 \rightarrow \epsilon / S_1 S_2$$

$S =$ start symbol

Here, S is the start symbol of G . The production rule $S \rightarrow \epsilon / S_1 S_2$ tells S can be replaced by ϵ or $S_1 S_2$. i.e. by using $S \rightarrow \epsilon$, it can generate the empty string and $S \rightarrow S_1 S_2$, it can generate replace S by $S_1 S_2$. First it uses production rule R , and generates all string of G_2 number of times.

① Design a CFG that generates all the palindrome over $\Sigma = \{a, b\}$.

② Design a CFG for the language $L = \{a^n b^n : n > 0\}$

$$\rightarrow S \rightarrow a S a / b S b / \epsilon$$

$$G = (V, \Sigma, R, S)$$

where,

$$V = \{S\}$$

$$\Sigma = \{a, b\}$$

R consists of -

$$S \rightarrow a S a / b S b / \epsilon$$

$S =$ start symbol

$$w = 101101$$

$$S \rightarrow 1 S 1$$

$$\rightarrow 10 S 1$$

$$\rightarrow 10 S 101$$

$$\rightarrow 101 S 101$$

$$\rightarrow 101101$$

$$w = 010$$

$$S \rightarrow 0 S 0$$

$$\rightarrow 010$$

2.

$$\rightarrow G = (V, \Sigma, R, S)$$

where,

$$V = \{S, A, B\}$$

$$\Sigma = \{a, b\}$$

R consists of

$$S \rightarrow A B$$

$$A \rightarrow a A / a$$

$$B \rightarrow b B / b$$

$S =$ start symbol

$s \rightarrow asb$
 $\rightarrow aabb$

$s \rightarrow asb$
 $\rightarrow aasbb$
 $\rightarrow aaabbb$

Let $w = a^n 1^n$ $w^R = 1^n a^n$
 $s \rightarrow AB$
 $A \rightarrow 0A1/01$
 $B \rightarrow 1B0/10$

Let $L = \{a^n b^n : n > 0\}$

$s \rightarrow ~~asbb~~ asbb | abb$

$\rightarrow$ Soln:
 $G = (V, \Sigma, R, s)$

where,

$V = \{s\}$

$\Sigma = \{a, b\}$

R consists of-

$s \rightarrow asbb | abb$

Examples: $aaabbbbbbb$

$s \rightarrow asbb$

$\rightarrow aasbbbb [s \rightarrow asbb]$

$\rightarrow aaabbbbbbb [s \rightarrow abb]$

Pushdown automata:

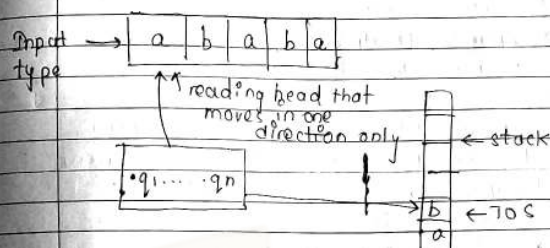


Fig: Block diagram of a PDA

Mathematically a PDA is defined as:

$M = (Q, \Sigma, \delta, q_0, F, \Gamma)$

where,

Q = finite set of states

Σ = finite set of inputs

δ = transition function

q_0 = initial state

F = set of final state

Γ = stack symbols

Move of a PDA

- The instant given by a transition function of a PDA is called its move.

- A move of a PDA is defined as:

$\delta(q, x, Y) \rightarrow (p, z) \Rightarrow [Q \times \Sigma \times \Gamma \rightarrow Q \times \Gamma]$

1.2	.3849	.3865	.3886	.3908	.3929	.3949	.3970	.3990	.4010	.4021
1.3	.4032	.4049	.4066	.4082	.4099	.4115	.4131	.4147	.4162	.4177
1.4	.4192	.4207	.4222	.4236	.4251	.4265	.4279	.4292	.4306	.4319
1.5	.4332	.4345	.4357	.4370	.4382	.4394	.4406	.4418	.4429	.4441
1.6	.4452	.4463	.4474	.4484	.4495	.4505	.4515	.4525	.4535	.4545
1.7	.4554	.4564	.4573	.4582	.4591	.4599	.4608	.4616	.4624	.4632
1.8	.4641	.4649	.4656	.4664	.4671	.4678	.4685	.4692	.4699	.4706
1.9	.4713	.4719	.4726	.4732	.4738	.4744	.4750	.4756	.4762	.4768

where,

q = current state

x = symbol of input tape

y = current top of stack

p = next state

z = symbol that replaces the current top of the stack and becomes new top of stack.

eg. push 'a' to the empty stack.

$$\delta(q, a, \epsilon) \rightarrow (p, a)$$

$$\delta(q, a, a) \rightarrow (p, a, a)$$

II Pop 'a' from the stack

$$\delta(q, a, a) \rightarrow (p, \epsilon)$$

eg. Design a PDA for the language $L = \{a^n b^n : n > 0\}$

$$w = aabbbb$$

$$\textcircled{1} \delta(q, a, \epsilon) \rightarrow (f, a)$$

$$\textcircled{2} \delta(f, a, a) \rightarrow (f, a, a)$$

$$\textcircled{3} \delta(f, b, a) \rightarrow (f, \epsilon)$$

Let $M = (Q, \Sigma, \delta, q_0, F, \Gamma)$ be the PDA.

where,

$$Q = \{q, f\}$$

$$\Sigma = \{a, b\}$$

δ consists of

$$\textcircled{1} \delta(q, a, \epsilon) \rightarrow (f, a)$$

$$\textcircled{2} \delta(f, a, a) \rightarrow (f, a, a)$$

$$\textcircled{3} \delta(f, b, a) \rightarrow (f, \epsilon)$$

$$q_0 = q$$

$$F = f$$

$$\Gamma = \{a\}$$

$$w = aabbbb$$

State	Unread input	Stack Content	Transition used
q	aabbbb	empty	-
f	aabbbb	a	(1)
f	abbbb	aa	(2)
f	bbb	aaa	(2)
f	bb	aa	(3)
f	b	a	(3)
f	e	e	(3)

Since, the string & stack both becomes empty, the string is accepted.

1.2	.3849	.3869	.3888	.3907	.3925	.3944	.3962	.3980	.3997	.4015
1.3	.4032	.4049	.4066	.4082	.4099	.4115	.4131	.4147	.4162	.4177
1.4	.4192	.4207	.4222	.4236	.4251	.4265	.4279	.4292	.4306	.4319
1.5	.4332	.4345	.4357	.4370	.4382	.4394	.4406	.4418	.4429	.4441
1.6	.4452	.4463	.4474	.4484	.4495	.4505	.4515	.4525	.4535	.4545
1.7	.4554	.4564	.4573	.4582	.4591	.4599	.4608	.4616	.4625	.4633
1.8	.4641	.4649	.4656	.4664	.4671	.4678	.4685	.4692	.4699	.4706
1.9	.4713	.4720	.4727	.4734	.4741	.4748	.4755	.4762	.4769	.4776

* Acceptance of a string by PDA:

i) Acceptance by final state

ii) Acceptance by empty **stack**

A string is said to be accepted if the stack becomes empty, when the string terminates.

Q. Design a PDA for the language $L = \{ \text{set of all the string over } \Sigma = \{a, b\} \text{ that contains equal number of } a\text{'s and } b\text{'s} \}$

→ Let $M = (Q, \Sigma, \delta, q_0, F, \Gamma)$ be the PDA.
Where,

$$Q = \{q, f\}$$

$$w = ababbb$$

$$\Sigma = \{a, b\}$$

δ consists of

- ① $a \rightarrow \delta(q, a, \epsilon) \rightarrow (f, a)$
- ② $aa \rightarrow \delta(f, a, a) \rightarrow (f, aa)$
- ③ $b \rightarrow \delta(q, b, \epsilon) \rightarrow (f, b)$
- ④ $bb \rightarrow \delta(f, b, b) \rightarrow (f, bb)$
- ⑤ $ab \rightarrow \delta(f, b, a) \rightarrow (f, \epsilon)$
- ⑥ $ba \rightarrow \delta(f, a, b) \rightarrow (f, \epsilon)$

$$q_0 = q$$

$$F = \{f\}$$

$$\Gamma = \{q\}$$

state	unread input	stack content	Transition used
q	ababbb	empty	-
f	ababbb	a	①
q	babbb	a	② ⑤
f	abbb	a	①
f	bbb	aa	②
f	b	a	⑤
f	ϵ	ϵ	⑤

Q. Design a PDA for the language $L = \{a^n b^n; n > 0\}$
→ Let $M = (Q, \Sigma, \delta, q_0, F, \Gamma)$ be the PDA.

where,

$$Q = \{q, f\}$$

$$w = aabbbb$$

$$\Sigma = \{a, b\}$$

δ consists of

- ① $a \rightarrow \delta(q, a, \epsilon) \rightarrow (f, a)$ $bb \rightarrow$
- ② $b \rightarrow \delta(q, b, \epsilon) \rightarrow (f, b)$
- ③ $ab \rightarrow \delta(f, b, a) \rightarrow (f, \epsilon)$
- ④ $ba \rightarrow \delta(f, a, b) \rightarrow (f, \epsilon)$

$$q_0 = q, F = \{f\}, \Gamma = \{q\}$$

state	unread input	stack content	transition
q	aabbbb	empty	-
f	abbbb	a	①
f	bbbb	aa	②
f	bbb	a	③
f	bb	ϵ	④

- ① $\delta(q, a, \epsilon) \rightarrow (f, aa)$
- ② $\delta(q, a, a) \rightarrow (f, aaa)$
- ③ $\delta(q, f, b, a) \rightarrow (f, \epsilon)$

State	Unread input	Stack content	Transition used
q	aabbbb	empty	-
q	abbbb	aa	①
f	bbbb	aaaa	②
f	bbb	aaa	③
f	bb	aa	③
f	b	a	③
f	empty	empty	③

CFG to PDA Conversion:

- ① Push the start symbol to the empty stack
- ② Then, push the right side string of the start symbol to the stack.
- ③ If the top of stack is a non-terminal then push the right side of the string of non-terminal to the stack.
- ④ If the top of stack is terminal then read the symbol from the input string and pop the terminal from the stack.
- ⑤ Continue the process until all the symbols of input string are scanned.

eg:

$S \rightarrow aAs | os | b$
 $A \rightarrow oB | b$
 $B \rightarrow bA | a$

$w = aabbbab$
 $S \rightarrow aAs$
 $\rightarrow aabAs$
 $\rightarrow aabbAs$
 $\rightarrow aabbbaS$
 $\rightarrow aabbba$

- ① $\delta(q, \epsilon, \epsilon) \rightarrow (f, s)$
- ② $\delta(f, \epsilon, s) \rightarrow (f, aAs), (q, as), (f, b)$
- ③ $\delta(f, \epsilon, A) \rightarrow (f, aB), (f, b)$
- ④ $\delta(f, \epsilon, B) \rightarrow (f, bA), (f, a)$
- ⑤ $\delta(f, a, a) \rightarrow (f, \epsilon)$
- ⑥ $\delta(f, b, b) \rightarrow (f, \epsilon)$

Let $M = (Q, \Sigma, \delta, q_0, F, \Gamma)$ be a PDA

where,

$Q = \{q, f\}$

$\Sigma = \{a, b\}$

δ consists of -

$q_0 = q$

$F = f$

$\Gamma = \{s, A, B, a, b\}$

1.1	.3643	.3665	.3686	.3708	.3729	.3749	.3770	.3790	.3810	.3829
1.2	.3849	.3869	.3888	.3907	.3925	.3944	.3962	.3980	.3997	.4015
1.3	.4032	.4049	.4066	.4082	.4099	.4115	.4131	.4147	.4162	.4177
1.4	.4192	.4207	.4222	.4236	.4251	.4265	.4279	.4292	.4306	.4319
1.5	.4332	.4345	.4357	.4370	.4382	.4394	.4406	.4418	.4429	.4441
1.6	.4452	.4463	.4474	.4484	.4495	.4505	.4515	.4525	.4535	.4545
1.7	.4554	.4564	.4573	.4582	.4591	.4599	.4608	.4616	.4625	.4634
1.8	.4641	.4649	.4656	.4664	.4671	.4678	.4685	.4692	.4699	.4706
1.9	.4713	.4719	.4726	.4732	.4738	.4744	.4750	.4756	.4762	.4768

state	unreadinput	stackcontent	transition used
q	ababb	empty	①
f	ababb	s	②
f	ababb	as	③
f	ababb	as	④
f	ababb	as	⑤
f	ababb	as	⑥
f	ababb	as	⑦
f	ababb	as	⑧
f	ababb	as	⑨
f	ababb	as	⑩
f	ababb	as	⑪
f	ababb	as	⑫
f	ababb	as	⑬
f	ababb	as	⑭
f	ababb	as	⑮
f	ababb	as	⑯
f	ababb	as	⑰
f	ababb	as	⑱
f	ababb	as	⑲
f	ababb	as	⑳
f	ababb	as	㉑
f	ababb	as	㉒
f	ababb	as	㉓
f	ababb	as	㉔
f	ababb	as	㉕
f	ababb	as	㉖
f	ababb	as	㉗
f	ababb	as	㉘
f	ababb	as	㉙
f	ababb	as	㉚
f	ababb	as	㉛
f	ababb	as	㉜
f	ababb	as	㉝
f	ababb	as	㉞
f	ababb	as	㉟
f	ababb	as	㊱
f	ababb	as	㊲
f	ababb	as	㊳
f	ababb	as	㊴
f	ababb	as	㊵
f	ababb	as	㊶
f	ababb	as	㊷
f	ababb	as	㊸
f	ababb	as	㊹
f	ababb	as	㊺
f	ababb	as	㊻
f	ababb	as	㊼
f	ababb	as	㊽
f	ababb	as	㊾
f	ababb	as	㊿

Construct the equivalent PDA of the following CFG
 $s \rightarrow aBB/1s$
 $B \rightarrow aA/1$
 $A \rightarrow 1B/0$
 Check whether the string $w = 100111 \in L(G)$

Let $M = (Q, \Sigma, \delta, q_0, f; \Gamma)$ be PDA

where,

$Q = \{q, f\}$
 $\Sigma = \{0, 1\}$

$s \rightarrow aBB/1s$
 $\rightarrow 10BB$
 $\rightarrow 100BBB$
 $\rightarrow 100111$

consists of -

- ① $\delta(q, \epsilon, \epsilon) \rightarrow (f, s)$
- ② $\delta(f, \epsilon, s) \rightarrow (f, 0BB), (f, 1s)$
- ③ $\delta(f, \epsilon, A) \rightarrow (f, 1B), (f, 0)$
- ④ $\delta(f, \epsilon, B) \rightarrow (f, 0A), (f, 1)$
- ⑤ $\delta(f, 1, a) \rightarrow (f, \epsilon)$
- ⑥ $\delta(f, 0, o) \rightarrow (f, \epsilon)$

state	unreadinput	stackcontent	transition used
q	100111	empty	-
f	100111	s	①
f	100111	1s	②
f	00111	s	③
f	00111	0BB	④
f	0111	BB	⑤
f	0111	0BBB	⑥
f	111	BBB	⑦
f	111	1BB	⑧
f	11	BB	⑨
f	11	1B	⑩
f	1	B	⑪
f	1	1	⑫
f	1	1	⑬
f	1	1	⑭
f	1	1	⑮
f	1	1	⑯
f	1	1	⑰
f	1	1	⑱
f	1	1	㉑
f	1	1	㉒
f	1	1	㉓
f	1	1	㉔
f	1	1	㉕
f	1	1	㉖
f	1	1	㉗
f	1	1	㉘
f	1	1	㉙
f	1	1	㉚
f	1	1	㉛
f	1	1	㉜
f	1	1	㉝
f	1	1	㉞
f	1	1	㉟
f	1	1	㊱
f	1	1	㊲
f	1	1	㊳
f	1	1	㊴
f	1	1	㊵
f	1	1	㊶
f	1	1	㊷
f	1	1	㊸
f	1	1	㊹
f	1	1	㊺
f	1	1	㊻
f	1	1	㊼
f	1	1	㊽
f	1	1	㊾
f	1	1	㊿

1.2	.3849	.3865	.3686	.3708	.3729	.3749	.3770	.3790	.3810	.3621
1.3	.4032	.4049	.4066	.4082	.4099	.4115	.4131	.4147	.4162	.4015
1.4	.4192	.4207	.4223	.4236	.4251	.4265	.4279	.4292	.4306	.4177
1.5	.4332	.4345	.4357	.4370	.4382	.4394	.4406	.4418	.4429	.4319
1.6	.4452	.4463	.4474	.4484	.4495	.4505	.4515	.4525	.4535	.4441
1.7	.4554	.4564	.4573	.4582	.4591	.4599	.4608	.4617	.4625	.4545
1.8	.4641	.4649	.4656	.4664	.4671	.4678				
1.9	.4713	.4719	.4726	.4732	.4738	.4744				

* Pumping lemma for CFL

Let L be a context free language and $z \in L$ such that $|z| \geq n$. Then, z can be decomposed into sub-string u, v, w, x and y such that -

- (1) $|vwx| \geq 1$
- (2) $|vwx| \leq n$
- (3) $uv^kwx^ky \in L$ for all $k \geq 0$.

Proof:

Let us consider a CFG -

$S \rightarrow AB$

$A \rightarrow aB|a$

$B \rightarrow bA|b$

Let L be a CFL and $z \in L$.

Now, $z = ababb \in L$

Then, according to pumping lemma, we can decompose z into sub-string u, v, w, x and y as -

$u = a$

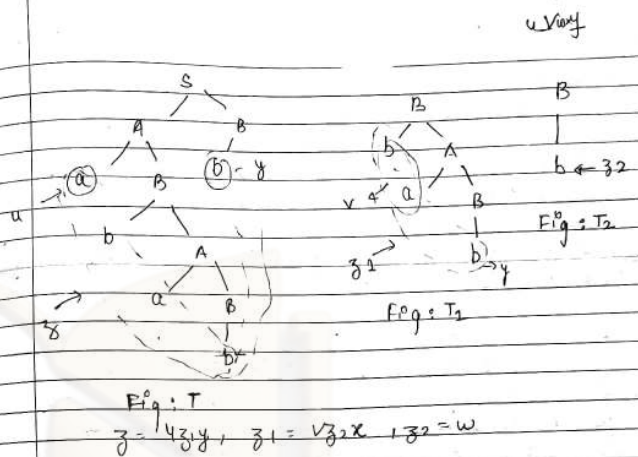
$v = ba$

$w = b$

$x = \epsilon$

$y = b$

Now, we construct a derivation tree for $z = ababb$ as -



Here, T_1 is a proper subtree of T and T_2 is a proper subtree of T_1 and z_1, z_2 and z_3 are yields of T_1, T_2 and T_3 respectively.

From, above derivation trees, we can write $z \rightarrow uv^kxy, z_1 \rightarrow vzx$ and $z_2 \rightarrow w$.

Since, T is a tree with root S and T_1 and T_2 are trees with root B , so we can write -

$S \rightarrow uBy, B \rightarrow vBx$ and $B \rightarrow w$

Now,

$S \rightarrow uBy$ gives $S \rightarrow uv^kwx^ky$, for $k=0$

$S \rightarrow uBy$ gives $S \rightarrow uvBxy$
 $\rightarrow uvwxxy \in L$, for $k=1$

$S \rightarrow uBy$ gives $S \rightarrow uvB^2xy$
 $\rightarrow uvvBxxy$

1.5	.4332	.4345	.4357	.4370	.4382	.4394	.4406	.4418	.4429	.4441
1.6	.4452	.4463	.4474	.4484	.4495	.4505	.4515	.4525	.4535	.4545
1.7	.4554	.4564	.4573	.4582	.4591	.4599	.4608	.4617		
1.8	.4641	.4649	.4656	.4664						
1.9	.4713	.4710			.4678					

→ $w^k x y$
→ $uv^2 w x^k y \in L$ for $k \geq 2^n$

Similarly, $uv^k w x^k y \in L$, for $k \geq 0$

eg Show that the language $L = \{a^n b^n c^n \mid n > 0\}$ is not context free.

→ Let L be context free language i.e. $L = \{a^n b^n c^n \mid n > 0\}$
and $w = a^p b^p c^p \in L$

According to the pumping lemma, w can be decomposed into three substrings such that u, v, w, x and y such that:

$$\begin{aligned} u &= a^r \\ v &= a^r, r > 0 \\ w &= a^p \\ x &= \epsilon \\ y &= a^{p-(q+r+s)} b^p c^p \end{aligned}$$

$$uv^2 wx^2 y = a^r a^{2r} a^p a^{p-(q+r+s)} b^p c^p = a^{p+r} b^p c^p$$

Since, $r > 0, p+r > p$

i.e. $a^{p+r} b^p c^p$ is not of the form $a^p b^p c^p$

$uv^2 wx^2 y \notin L$

Turing machine

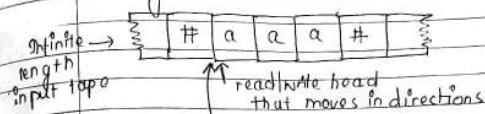


Fig: Block diagram of TM

A TM is defined as-

$$M = (Q, \Sigma, \delta, q_0, F, \Gamma, \#)$$

where,

Q = finite set of states

Σ = finite set of inputs

δ = transition function

q_0 = initial state

F = set of final states

Γ = symbols of input tape

$\#$ = blank symbol

the head of a TM can read from the input tape and can write the symbol back to the input tape. The write feature of a TM provides the facility of the storing the symbols permanently to the input tape.

1.3	.3899	.3869	.3888	.3907	.3925	.3944	.3962	.3980	.3997	.4015
1.4	.4032	.4049	.4066	.4082	.4099	.4115	.4131	.4147	.4162	.4177
	.4192	.4207	.4222	.4236	.4251	.4265	.4279	.4292	.4306	.4319
1.5	.4332	.4345	.4357	.4370	.4382	.4394	.4406	.4418	.4429	.4441
1.6	.4452	.4463	.4474	.4484	.4495	.4505	.4515	.4525	.4535	.4545
1.7	.4554	.4564	.4573	.4582	.4591	.4599	.4608			
1.8	.4641	.4649								
1.9										

further, it can read in both directions i.e. a TM can re-read the content of any cell at any time so that the result of previous calculations can be used for future processing. Due to these features a TM is said to be functionally stronger than PDA and FA.

Move of a TM:-

A move of TM is of the form.

$$\delta(q, x) \rightarrow (p, y, d)$$

where,

q = current state

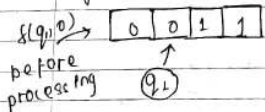
x = symbol of input tape or symbol under read/write head.

p = next state

y = symbol that replaces the symbol of the input tape.

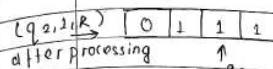
d = Direction of the next symbol to be read (L or R).

$$\text{eg: } \delta(q_0, 0) \rightarrow (q_1, 1, R)$$



Instantaneous description (ID)

$Q_1, 011$



$Q_1, q_2, 11$

$$Q_1, 011 \rightarrow Q_1, q_2, 11$$

eg

$$\delta(q, x_i) \rightarrow (p, y, L)$$

$$w = x_1 x_2 \dots x_i x_{i+1} \dots x_n$$

$$x_1 x_2 \dots x_{i-1} x_i x_{i+1} \dots x_n$$

before processing

$$x_1 x_2 \dots x_{i-1} q x_i x_{i+1} \dots x_n$$

after processing

$$x_1 x_2 \dots x_{i-1} y x_{i+1} \dots x_n$$

$$x_1 x_2 \dots x_{i-1} q x_i x_{i+1} \dots x_n \rightarrow x_1 x_2 \dots p x_{i-1} y x_{i+1} \dots x_n$$

Design of a TM:

- 1) Acceptance by final state
- 2) Acceptance by halt state

- Halt is a special state that is used to determine

1.5	.4332	.4345	.4357	.4370	.4382	.4394	.4406	.4418	.4429	.4441
1.6	.4452	.4463	.4474	.4484	.4495	.4505	.4515	.4525	.4535	.4545
1.7	.4554	.4564	.4573	.4582	.4591	.4599	.4608	.4617	.4625	.4633
1.8	.4641	.4649	.4656	.4663	.4670	.4677	.4684	.4691	.4698	.4705
1.9	.4711	.4718	.4725	.4732	.4739	.4746	.4753	.4760	.4767	.4774

the acceptance of a string.

- The difference between final state and halt state is that TM stops processing after transitioning to halt state.

* Design a TM that erases all the symbols of the strings over $\Sigma = \{a, b\}$.

$w = dbaab$

- ① $\delta(q_1, a) \rightarrow (q_2, \#, R)$
- ② $\delta(q_1, b) \rightarrow (q_2, \#, R)$
- ③ $\delta(q_2, a) \rightarrow (q_2, \#, R)$
- ④ $\delta(q_2, b) \rightarrow (q_2, \#, R)$
- ⑤ $\delta(q_2, \#) \rightarrow (q_1, \#, N)$

eg:

$q_1 dbaab \rightarrow \#q_2 dbaab$
 $\rightarrow \# \# q_2 aab$
 $\rightarrow \# \# \# q_2 ab$
 $\rightarrow \# \# \# \# q_2 b$
 $\rightarrow \# \# \# \# \# q_2 \#$
 $\rightarrow \# \# \# \# \# b$

* Design a TM that accepts all the strings of even length over $\Sigma = \{a, b\}$.

~~1. $\delta(q_1, a) \rightarrow (q_1, \#, R)$~~

~~2. $\delta(q_1, b) \rightarrow (q_1, \#, R)$~~

① $\delta(q_1, a) \rightarrow (q_2, a, R)$

$\delta(q_1, b) \rightarrow (q_2, b, R)$

$\delta(q_2, a) \rightarrow (q_1, a, R)$

$\delta(q_2, b) \rightarrow (q_1, b, R)$

$\delta(q_1, \#) \rightarrow (q_1, \#, N)$

eg:

1.5	.4332	.4345	.4357	.4370	.4382	.4394	.4406	.4418	.4429	.4441
1.6	.4452	.4463	.4474	.4484	.4495	.4505	.4515	.4525	.4535	.4545
1.7	.4554	.4564	.4573	.4582	.4591	.4599	.4608			
1.8	.4641	.4649	.4656	.4664	.4671	.4678				
1.9	.4713	.4719	.4726							

Turing machine for computing functions:

Let $f(w) = u$ be a function. Then f is said to be Turing computable if there exists a TM such that

$$\delta(q, \# w \#) \rightarrow (h, u)$$

When $n = 1$, $\#(f(n) = n+1), n > 0$

$$\begin{aligned} \#q_1 I \# &\rightarrow \# I q_2 \# \\ &\rightarrow \# I I q_2 \# \\ &\rightarrow \# I I I \# \end{aligned}$$

- ① $\delta(q_1, I) \rightarrow (q_1, I, R)$
- ② $\delta(q_1, \#) \rightarrow (q_2, I, R)$
- ③ $\delta(q_2, \#) \rightarrow (h, \#, N)$

When $n = 2$,

$$\begin{aligned} \#q_1 I I \# &\rightarrow \# I q_1 \# \\ &\rightarrow \# I I q_2 \# \\ &\rightarrow \# I I I \# \end{aligned}$$

When $n = 3$,

$$\begin{aligned} \#q_1 I I I \# &\rightarrow \# I q_1 I \# \\ &\rightarrow \# I I q_1 \# \\ &\rightarrow \# I I I q_2 \# \\ &\rightarrow \# I I I I \# \end{aligned}$$

Similarly,

$$\#q_1 I^n \rightarrow \dots \rightarrow \# I^{n+1} \#$$

$$\# f(n) = n+2, n > 0$$

When $n = 1$,

- ① $\delta(q_1, I) \rightarrow (q_1, I, R)$
- ② $\delta(q_1, \#) \rightarrow (q_2, I, R)$
- ③ $\delta(q_2, \#) \rightarrow (q_3, I, R)$
- ④ $\delta(q_3, \#) \rightarrow (h, \#, N)$

When $n = 2$,

$$\begin{aligned} \#q_1 I I \# &\rightarrow \# I q_1 \# \\ &\rightarrow \# I I q_2 \# \\ &\rightarrow \# I I I q_2 \# \\ &\rightarrow \# I I I I q_3 \# \\ &\rightarrow \# I I I I I \# \end{aligned}$$

When $n = 3$,

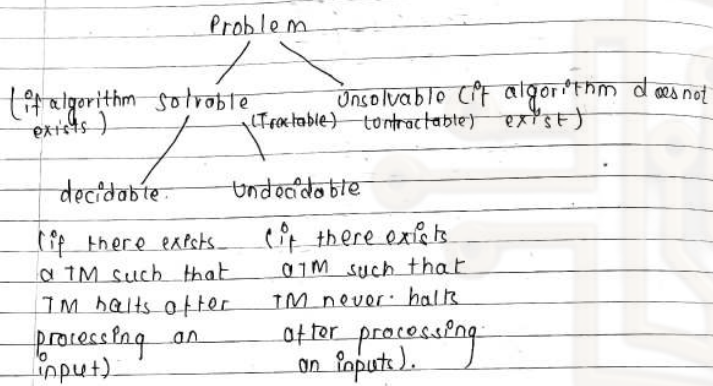
$$\begin{aligned} \#q_1 I I I \# &\rightarrow \# I q_1 I I \# \\ &\rightarrow \# I I q_1 I \# \\ &\rightarrow \# I I I q_2 \# \\ &\rightarrow \# I I I I q_3 \# \\ &\rightarrow \# I I I I I \# \end{aligned}$$

Similarly,

Language of TM

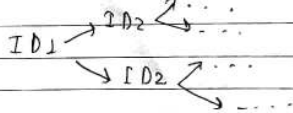
1. Recursive language (decidable problems)
2. Recursively enumerable language (decidable + undecidable problems)

Decidability and Undecidability



Types of TM:-

- ① Deterministic TM $\rightarrow TD_1 \rightarrow TD_2 \rightarrow \dots \rightarrow TD_n$
- ② Non-deterministic TM



class P, class NP problems:-

- A problem is said to be in class P if there exists a deterministic TM such that it can solve the problem in polynomial time.
eg. Kruskal's algorithm.
- A problem is said to be in class NP if there exists a non-deterministic TM such that it can solve the problem in polynomial time.
eg. Travelling Salesman problem.

Recursive function theory:-

- ① Partial function
- ② Total function
- ③ Initial function
 - ↳ zero functions
 - ↳ successor functions
 - ↳ projection functions
- ④ Recursion and composition
- ⑤ Primitive recursive functions.